L3. Introduction to Prediction Markets

Yurii Nosov

University of California, Los Angeles

UCLA Trustworthy AI Lab

Week 3

Before we start

This course is for research and education

The goal is fluency with prediction-market data for ML/AI analysis. Nothing in this lecture is trading advice, and you are never asked to trade.

Wallets are people

Wallet data in the course materials is **pseudonymized**. Do not try to identify the people behind any wallet, in the lab's data or anywhere else.

- Real markets, real trades and real wallets appear on these slides because that is what the data looks like. We study the **data**, not the traders.

Plan for today

1. **Prediction markets** 3:30 – 4:20
   - What Polymarket is, prices as probabilities, the order book, arbitrage
2. **Getting the data** 4:30 – 5:20
   - Where the data lives, the lab's database (catalog, trades, order books), a live walk-through in Python
3. **Using the data: insiders** 5:30 – 6:20
   - Confirmed cases, a first detector (preview of Week 5), what you could ask of the data

By 6:20 you should be able to

read any market on Polymarket, pull its data into Python, and say what an insider would look like in that data.

What is a prediction market?

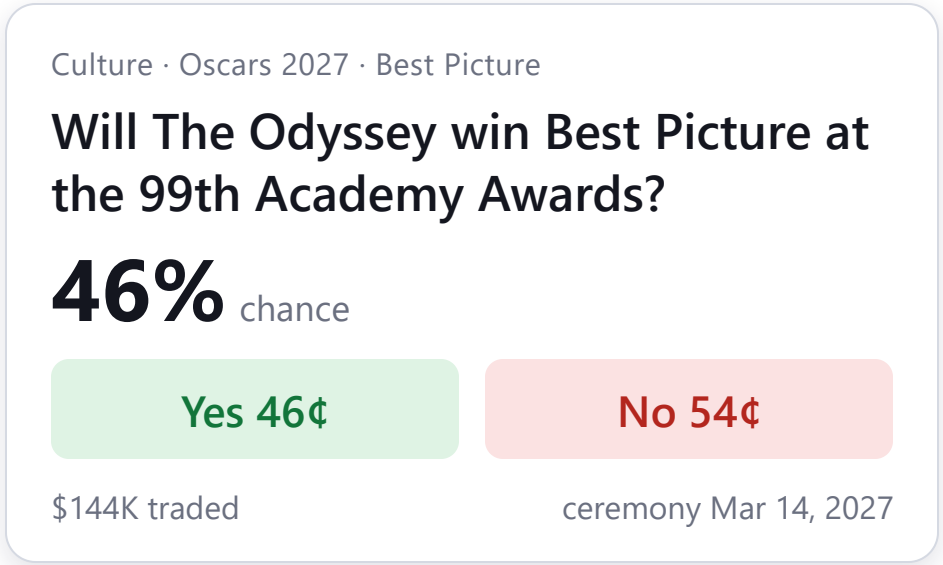
- A market where people trade contracts on the **outcome of a future event**.
- Each contract is a yes/no question with a deadline: *"Will X happen by date D?"*
- If the answer is Yes, a **Yes share** pays **$1**. Otherwise it pays **$0**.
- Until the question is resolved, shares trade at a price between $0 and $1.

Why a statistics course cares

The price is a number that a crowd produces about the future. It is public, timestamped, and updated every second. In other words: **data**.

Polymarket: the platform used in this course. Trades settle on a public blockchain (Polygon) in a dollar-pegged token. The lab's catalog holds 1,178,126 markets in 273,148 events. **[our database, Oct 7 2026]**

One market = one question



- **The question**, with a written rule for what counts as Yes.
- **Yes at 46¢**: the market's estimate is about 46%.
- **No at 54¢**: the other side of the same bet.
- **$144K traded**: how much money has changed hands.
- **An event** groups related markets: here, one market per film.

Card drawn from live data, not a screenshot. Prices are the midpoint; best bid 45¢, best ask 47¢. **[Polymarket API, Sep 30 2026]**

A share pays \$1 or \$0

What you buy	Cost today	If the answer is Yes	If the answer is No
1 Yes share	\$0.46	get \$1 (+\$0.54)	get \$0 (−\$0.46)
1 No share	\$0.54	get \$0 (−\$0.54)	get \$1 (+\$0.46)
1 Yes + 1 No	\$1.00	get \$1 (0)	get \$1 (0)

Key identity

$$\text{price(Yes)} + \text{price(No)} = \$1$$

One Yes share plus one No share is a guaranteed dollar, whatever happens.

- When the two prices do not add up to \$1, the identity is violated and the market is mispriced. That is **arbitrage**, later in this block.

Why the price is a probability

- You believe the event happens with probability p. A Yes share costs c.
- Your expected profit from buying one share:

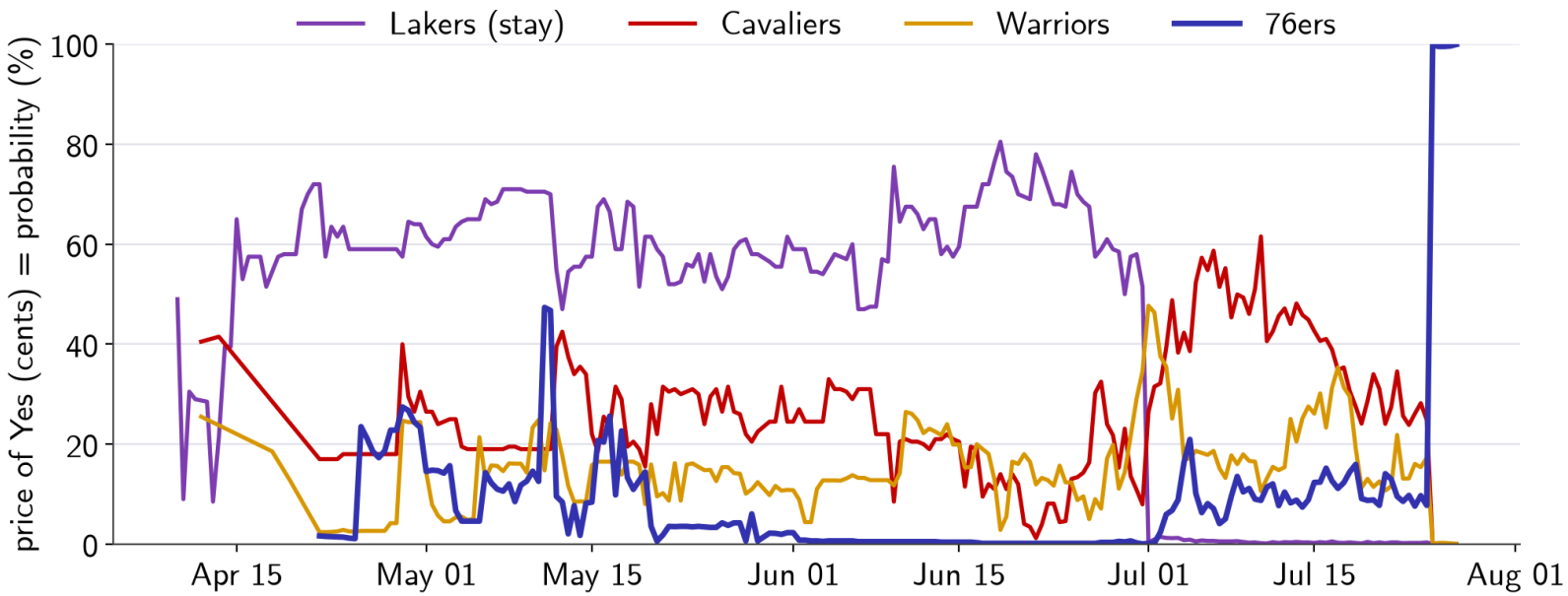
$$E[\text{profit}] = p\,(1 - c) + (1 - p)\,(0 - c) = p - c$$

- If $p > c$ you buy, and buying pushes the price up. If $p < c$ you sell, and the price falls.
- The price rests where buyers and sellers balance: $c \approx$ the crowd's p.

Where this bends

Fees, thin markets, risk-averse traders, and money locked up until the deadline all push the price away from the true probability. Read a price as an **estimate**, not a fact.

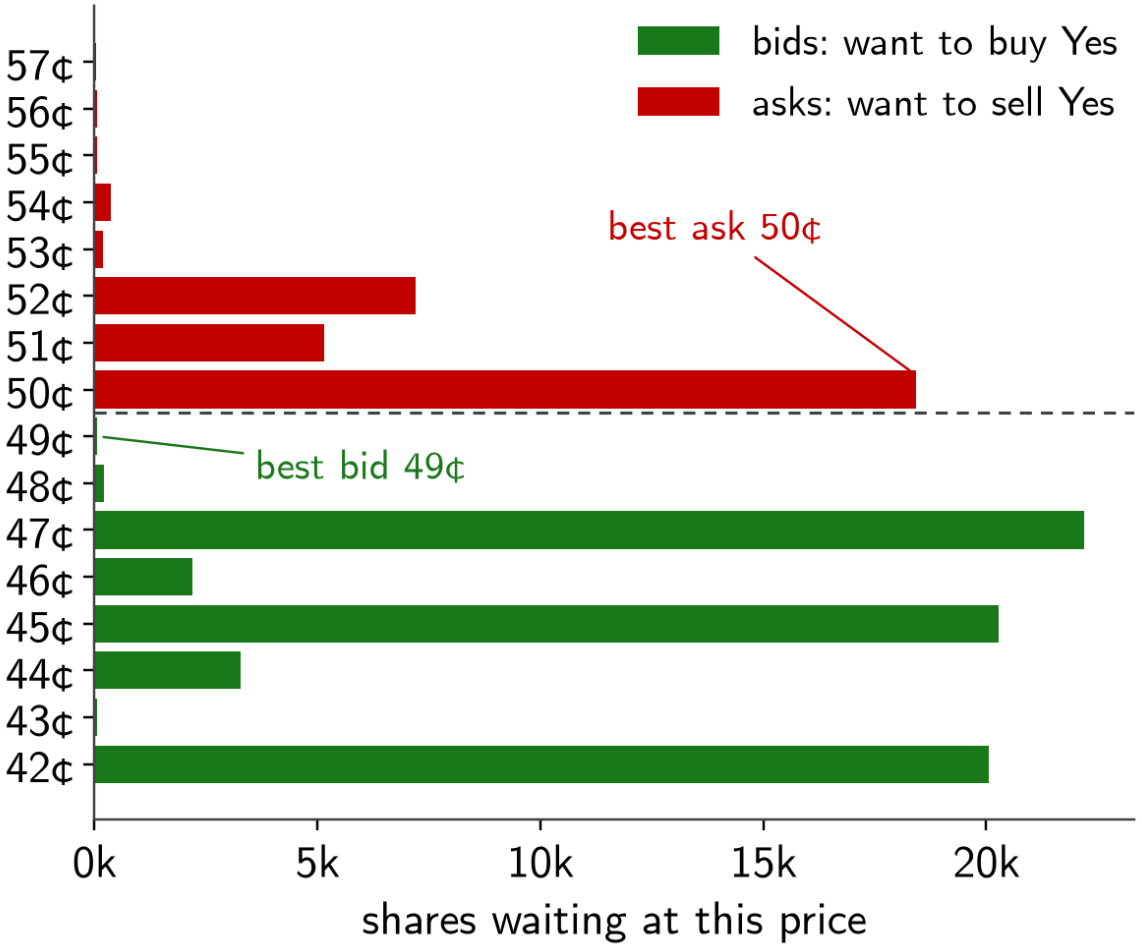
A market from birth to resolution



- One market per possible answer: *"Which team will LeBron James play for next?"* Each line is a price.
- The answer, the **76ers**, traded near **2%** for all of June and **under 10%** just before the news.
- Someone bought those shares at 8¢ and was paid $1. Lucky, skilled, or informed? That is Block 3.

36 markets in the event, $46.2M traded, resolved Jul 27 2026. Prices every 12 hours. **[Polymarket price-history API]**

Where the price comes from: the order book



- Nobody sets the price. Traders post offers, and the list of offers is the **order book**.
- **Bid**: "I will buy Yes at this price."
  Ask: "I will sell Yes at this price."
- **Spread** = best ask − best bid. Here 50¢ − 49¢ = 1¢.
- The "price" you see quoted is the middle of the spread, or the last trade.

Market: "Will Bitcoin reach $95,000 by December 31, 2026?", $767K traded. Top 8 levels on each side. **[Polymarket order-book API, Sep 30 2026]**

Makers, takers, and one worked order

- A **maker** posts an offer and waits.
- A **taker** accepts an offer that is already there.
- Every trade has one of each, and the blockchain record of the trade names **both wallets** (slide 22).

A taker buys $12,000 of Yes	**Shares**	**Price**	**Cost**
takes everything at the best ask	18,434	50¢	$9,217
then the next level	5,151	51¢	$2,627
then part of the next	300	52¢	$156
total	**23,885**	**50.24¢**	**$12,000**

A large order moves the price

This one order pushes the best ask from 50¢ to 52¢. In a thin market, a big bet is **visible**: it leaves a mark on the price and on the book. Keep that in mind for Block 3.

Who decides the answer?

1. The event happens
The deadline passes, or the outcome becomes known early.

→

2. A proposal
Anyone may propose the answer, and posts a bond ($500 in the markets shown today; set per market).

→

3. Challenge window
2 hours in which anyone may dispute the proposal.

→

4. Resolved
Each winning share is redeemed for $1. Losing shares are worth $0.

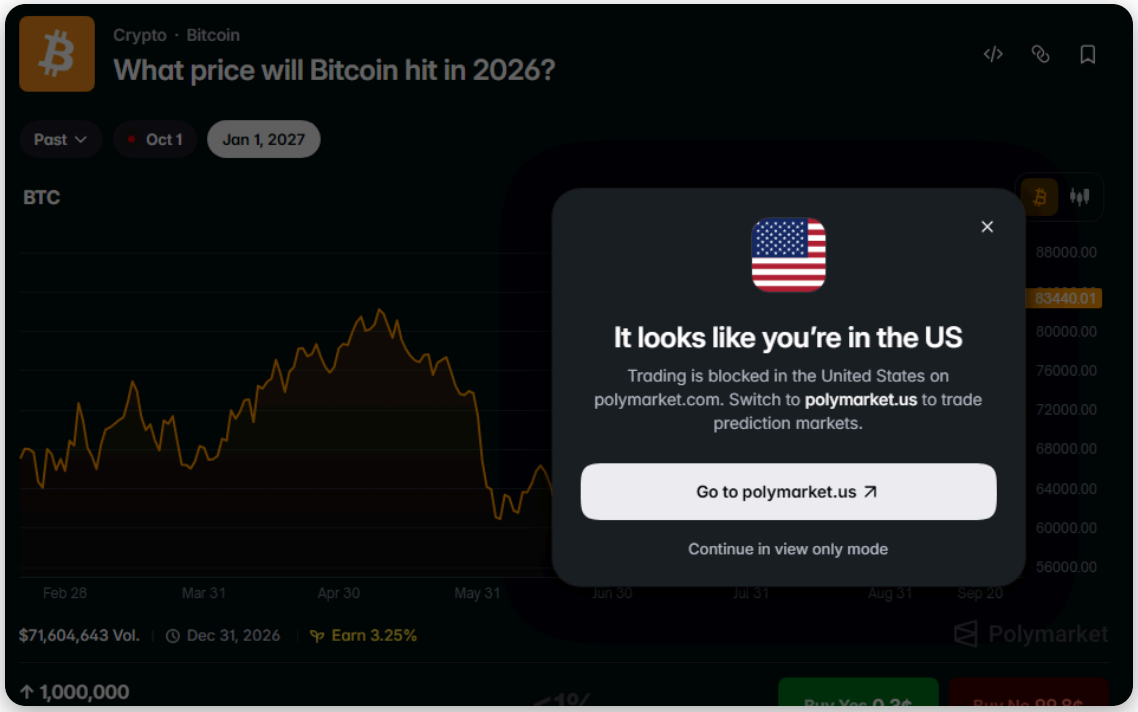
If disputed: a second proposal round. If disputed again, holders of the UMA token vote (UMA is the independent "oracle" service that Polymarket outsources resolution to), which takes about 48 hours. The side that was wrong loses its bond.

Read the rule, not the headline

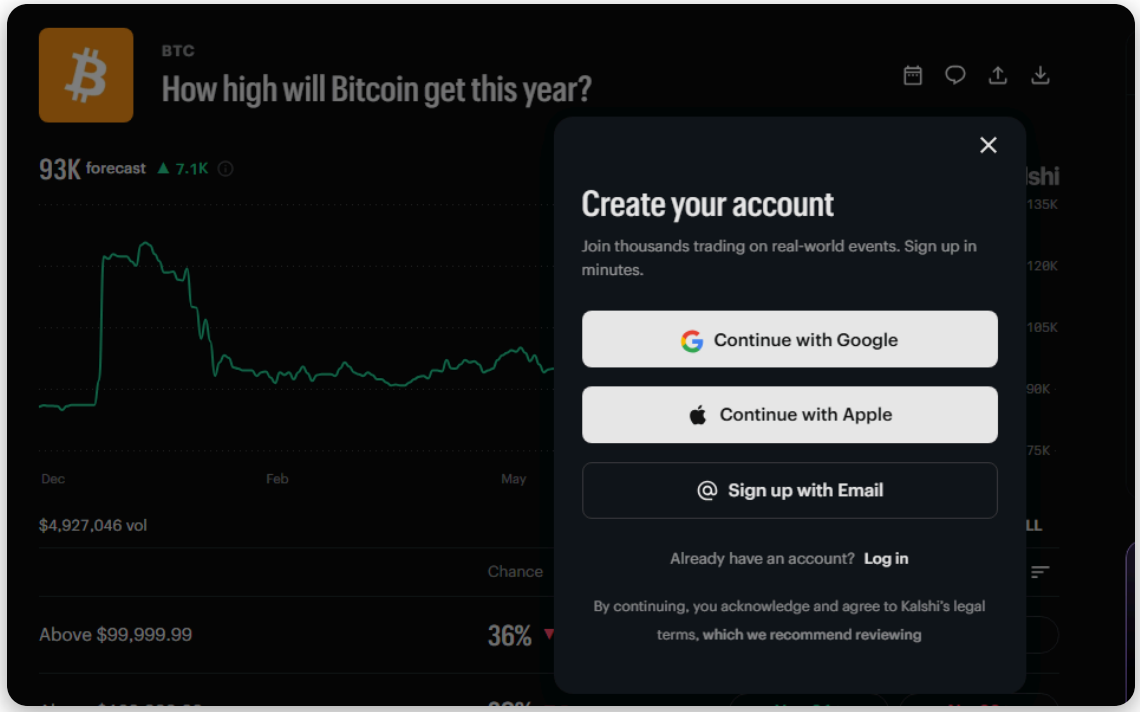
"Will Bitcoin reach $95,000?" resolves Yes if *any one-minute candle on Binance* has a high of $95,000 or more. A market is a bet on its written rule, and two similar-looking markets can have different rules.

Resolution runs through the UMA optimistic oracle; no company employee picks the answer. **[Polymarket documentation, docs.polymarket.com/concepts/resolution]**

Two platforms, two front doors



Polymarket (polymarket.com). From the US: view only.
No account is needed to look, or to download the data.



Kalshi (kalshi.com). Open to the US.
You trade from a verified account, like a brokerage.

Both pages show the same kind of question: what price will Bitcoin hit this year? **[screenshots taken from Los Angeles, Sep 30 2026]**

Polymarket vs Kalshi

	Polymarket	**Kalshi**
Where trades settle	a public blockchain (Polygon)	the exchange's own ledger
Money	a dollar-pegged crypto token	US dollars
Who you are	a wallet address, no name attached	a verified person (ID checked)
Regulation	main site offshore; a separate US site registered with the CFTC	US exchange regulated by the CFTC
Who resolves	an open oracle with bonds and disputes	the exchange, by its rulebook
What the public sees	**every trade, with wallet addresses**	every trade, no identities

Same contract, opposite transparency

Kalshi knows every trader and publishes none of them. Polymarket publishes every wallet and knows none of them. For the course: either platform for the midterm; wallet-level work needs Polymarket; a Kalshi team works at the market level, price and volume moves before the news.

CFTC = Commodity Futures Trading Commission, the US regulator for these contracts. **[public record; regulation rows as of mid-2026]**

The same kind of question from two APIs

Polymarket


```
{
  "question": "Will Bitcoin reach $95,000
               by December 31, 2026?",
  "outcomes": ["Yes", "No"],
  "outcomePrices": ["0.495", "0.505"],
  "bestBid": 0.49,  "bestAsk": 0.5,
  "volume": "767423.65",
  "endDate": "2027-01-01T05:00:00Z",
  "conditionId": "0xacec7e5b…2dca784",
  "clobTokenIds": ["37011650…", "10149205…"]
}
```


Kalshi


```
{
  "title": "Will Bitcoin be above $99,999.99
            by Dec 31, 2026 at 11:59 PM ET?",
  "ticker": "KXBTCMAXY-26DEC31-99999.99",
  "yes_bid_dollars": "0.3500",
  "yes_ask_dollars": "0.3600",
  "volume_fp": "1530125.25",
  "close_time": "2027-01-01T04:59:00Z",
  "rules_primary": "If the Bitcoin spot price
     according to the CF Bitcoin Real-Time
     Index is above $99999.99 … resolves to Yes."
}
```


- Both are free and need no key. Highlighted: `conditionId` is the market's id on the blockchain, `clobTokenIds` are the ids of its Yes and No shares. Every later data source is joined on them. Kalshi has a ticker instead.

Real responses, shortened. Polymarket returns 89 fields per market, Kalshi 43. **[gamma-api.polymarket.com and api.elections.kalshi.com, Sep 30 2026]**

One trade, as each platform reports it

Polymarket


```
{
  "proxyWallet": "0x1e31c6b0…7e58a2db",
  "side": "BUY",   "outcome": "No",
  "size": 9.615385,
  "price": 0.52,
  "timestamp": 1790729493,
  "transactionHash": "0xd209e52b…21ce2728"
}
```


Kalshi


```
{
  "trade_id": "07233278-82e9-…-fa94596a4e1e",
  "ticker": "KXBTCMAXY-26DEC31-99999.99",
  "taker_side": "yes",
  "count_fp": "78.66",
  "yes_price_dollars": "0.3600",
  "created_time": "2026-09-30T05:15:34Z"
}
```


The difference that makes Block 3 possible

A Polymarket trade names a **wallet** (`proxyWallet`: the address Polymarket creates for each user) and a **transaction** anyone can look up. A Kalshi trade names no one. Insider detection from public data is only possible on the first.

The latest trade in each Bitcoin market at the time of the request. Addresses shortened. **[data-api.polymarket.com and api.elections.kalshi.com, Sep 30 2026]**

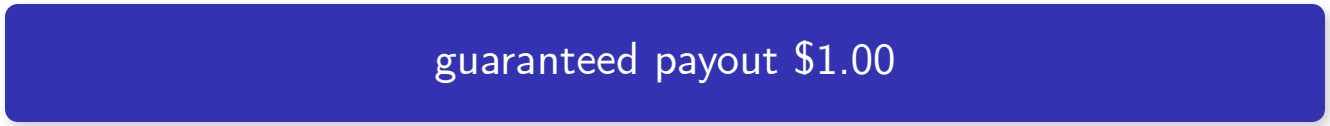
Arbitrage 1: inside one market

- Recall: one Yes + one No share always pays **$1**.
- If you can *buy* both for less than $1, the difference is profit with no risk:

what you pay: 95¢



what you get: $1, whatever happens



- Our Bitcoin market right now: 50¢ + 51¢ = $1.01. No violation; the extra cent is the spread.

What the lab measured

One month of NBA markets (Feb–Mar 2026):

173 games, 3,042 markets.

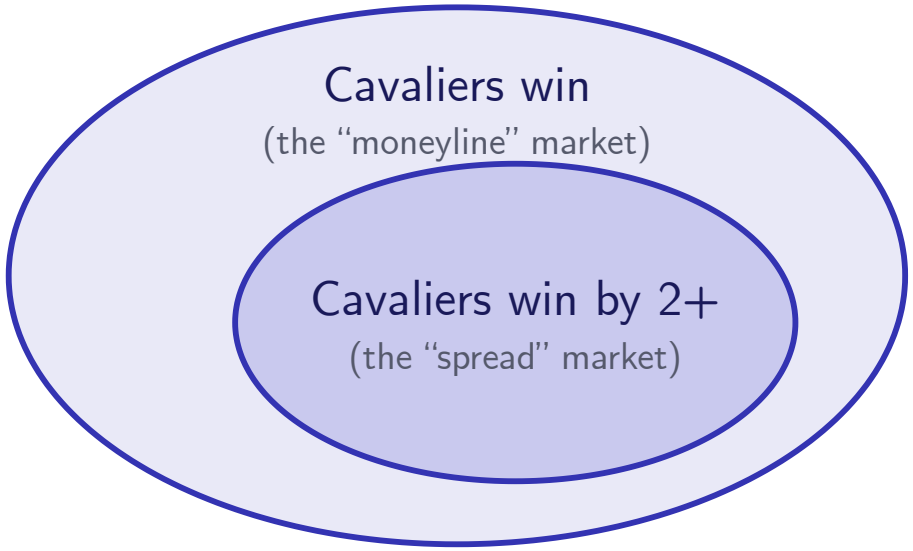
7 violations found

median lifetime **3.6 seconds**

gone before a human could click

Episodes counted at the top of the book with at least $10 available, from recorded order books. A statement about market efficiency, testable on other categories. **[lab result, 2026]**

Arbitrage 2: across two markets



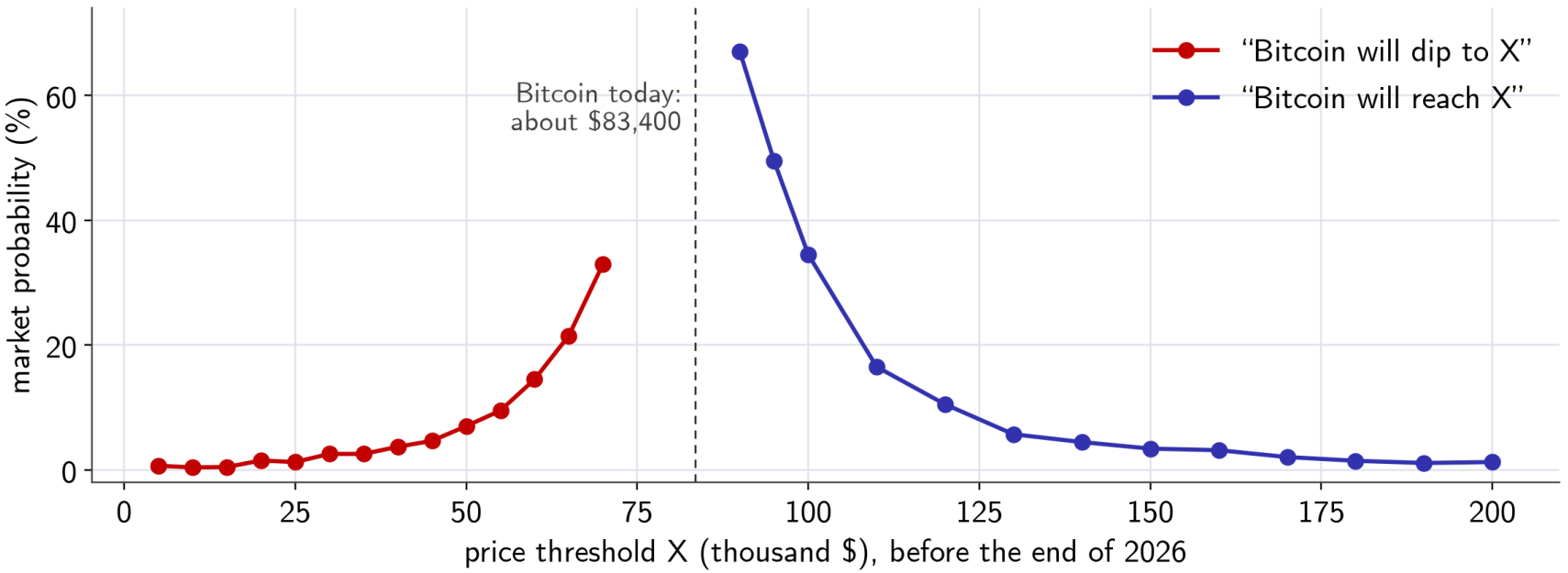
$$P(\text{win by } 2+) \le P(\text{win})$$

- One event lies inside the other, so its price must be lower. Sometimes it is not: here "win by 2+" traded at **64¢**, above the **60¢** of "win".
- Then buy **Cavaliers win** at 60¢ and **No** on the spread market (Cavaliers do *not* win by 2+) at 36¢. Cost: 96¢.

Final score	**First bet**	**Second bet**	**You get**
Cavaliers by 2+	\$1	\$0	**\$1**
Cavaliers by 1	\$1	\$1	**\$2**
Magic win	\$0	\$1	**\$1**

170 NBA games, Feb 4 – Mar 4, 2026: 290 such moments, median lifetime 11 s before the game and 16 s during it; most too thin to act on. The statistical idea is monotonicity: A ⊆ B implies P(A) ≤ P(B). **[lab result, 2026]**

Markets are not independent



- One event, **30 open markets** (47 counting the resolved ones): a bet for every price level. Together they trace a **whole distribution**.
- Each curve must be monotone. A point out of line is the arbitrage of the previous slide.
- One piece of news moves all thirty at once. They are not 30 independent samples.

Event "What price will Bitcoin hit in 2026?", $71.6M traded. Each dot is one market's Yes price. **[Polymarket API, Sep 30 2026]**

What to take from Block 1

1. A share pays **\$1 or \$0**. Yes and No together are worth exactly \$1.
2. The price is the crowd's **estimate of a probability**. Useful, and imperfect.
3. Prices come from an **order book**. Large orders move them and leave a trace.
4. A market is a bet on its **written rule**. Read the rule first.
5. Related markets are tied together. Violations are **arbitrage** and last seconds.
6. Polymarket publishes **every trade with a wallet address**. That is our raw material.

After the break (4:30)

Where this data lives, how the lab collects it, and how you get it into Python.

Block 2: how we get the data, and how you will

1. **Where Polymarket data lives**: three layers, one real trade seen in each.
2. **What vanishes**: the order book exists only in the moment.
3. **The lab's database**: a catalog of every market and who won; the order books of about 30,000 tracked markets, hour by hour, since May 30; and **every trade with a pseudonymous wallet**.
4. **Live**: eight questions to the database in Python; then an order book rebuilt tick by tick, on slides.
5. **The quirks**: three things the documentation does not say.
6. **From a category to its wallets**, and **validate what you collect**.

By 5:20 you should be able to

open the notebook, connect, find any market, pull its trades and its order book for any hour, and know what this database can and cannot tell you.

Three layers of Polymarket data

1 **The catalog**	What markets exist: question, rule, dates, ids, current price and volume. One row per market.	`gamma-api.polymarket.com`
2 **Prices and trades**	What happened in a market: every trade (with a wallet), the price history, and the order book *right now*.	`clob.polymarket.com` `data-api.polymarket.com`
3 **The chain**	Which wallet did it: every trade is a transaction on Polygon, permanent and public, between two wallet addresses.	`Polygon blockchain` `(any node or explorer)`

Where the lab's database sits

It stores **layer 1** (the whole catalog, plus who won), **layer 2's trades** with the wallet replaced by a pseudonym, and the one thing layer 2 forgets: the **order book over time**. Layer 3 stays on the chain; this course does not need it.

All three layers are public and free. Layers 1 and 2 are HTTP APIs that return JSON. Layer 3 is a blockchain: read it through a public node or an explorer such as polygonscan.com. **[Polymarket documentation, docs.polymarket.com]**

One trade seen at all three layers

1. Catalog row (our database)

title	Will Bitcoin reach $95,000 by December 31, 2026?
condition_id	0xacec7e5b…2dca784
token_ids	[37011650…, 10149205…]
end_date	2027-01-01 05:00 UTC
last_trade	0.34 (synced Sep 12)

2. Trade record (data API)

proxyWallet	0x1e31c6b0…7e58a2db
side / outcome	BUY No
size / price	9.615385 @ 0.52
timestamp	2026-09-30 00:51:33 UTC
transactionHash	0xd209e52b…21ce2728

3. The transaction on Polygon

block	94,682,902 · 2026-09-30 00:51:33 UTC
status	success · 20 events logged
to	0xe1111800…7b996b *Polymarket exchange contract*
0x1e31c6b0… paid	$5.19 *taker: 9.615385 × 0.52 = $5.00, plus $0.19 fee*
0xd1efe8a5… paid	$4.62 *maker: 9.615385 × 0.48*
minted	9.615385 Yes + 9.615385 No *on condition 0xacec7e5b…*
delivered	Yes → 0xd1efe8a5… No → 0x1e31c6b0…

- The same **condition id** and the same **wallet** appear in all three layers. Those two ids are how you join them.

The latest trade of the Bitcoin market at the time of the request, followed to the chain through its transaction hash. The two sides paid $9.81 for $9.62 of shares; the difference is the fee. Read the chain yourself: paste the hash into polygonscan.com, or ask any public Polygon node for `eth_getTransactionReceipt`. Addresses shortened. **[our database; data-api.polymarket.com; Polygon public RPC, Sep 30 2026]**

What vanishes: the order book exists only in the moment

Endpoint	Remembers
trades	everything, since the market opened
prices-history	the price at chosen intervals
book	**only right now**

- Ask for the 05:50 book and you get the book of the moment you ask. The 05:50 book is gone.
- Anything about **liquidity**, **spreads** or **arbitrage** over time needs someone to have been listening.

Same market, one hour apart

Bitcoin $95k, Yes	05:50 UTC	06:53 UTC
ask at 50¢	18,434 shares	23,428 shares
ask at 51¢	5,151	2,266
ask at 52¢	7,205	3,220
bid at 48¢	210	85
bid at 47¢	22,202	20,202

The left column exists only because we saved it.

So the lab records it

A recorder built by June, a lab member, subscribes to Polymarket's live feed and writes every change to every tracked order book, hour by hour, since **May 30, 2026**.

Book excerpts are the same market as slides 9 and 10, fetched twice on Sep 30 2026. Polymarket does not replay past order-book messages, so a gap in recording is permanent. **[clob.polymarket.com/book; June's data guide]**

The lab's database, part 1: the catalog

- A **PostgreSQL** database, read-only, one shared login. You talk to it in **SQL**; every query comes back as a pandas table.
- **`markets`**: one row per market, **1,178,126** rows.
- **`events`**: one row per event, **273,148** rows. An event groups markets (one Bitcoin event holds 47).
- **`resolutions`**: one row per **resolved** market, **213,602** since Jan 2026: who won, at what price, when. Joins on the slug.
- **Keys:** `slug` joins the three tables and the trade files; `condition_id` and `token_ids` open the order books; `synced_at` says when the row was refreshed.

One row of `markets`

slug	will-bitcoin-reach-95000-by-december-31-2026-…
title	Will Bitcoin reach $95,000 by December 31, 2026?
condition_id	0xacec7e5b90ccb1ac4c950e3aff18156d6ce1bb7b0c…
synced_at	1789248242 = 2026-09-12 21:24 UTC
start_date	2026-06-08 04:57 UTC
end_date	2027-01-01 05:00 UTC
active / closed	True / False
neg_risk	False
last_trade_price	0.34
best_bid	0.32
best_ask	0.34
volume_clob	387913.75
volume_24hr	2618.21
volume_1wk	170995.62
token_ids	[37011650970792437921…, 10149205635175183392…]
outcomes	[Yes, No]
event_slug	what-price-will-bitcoin-hit-before-2027

Counts as of Oct 7 2026. New rows reach `resolutions` about 15 minutes after a market closes. A fourth table, `market_hourly`, is still empty. **[our database; June's data guide]**

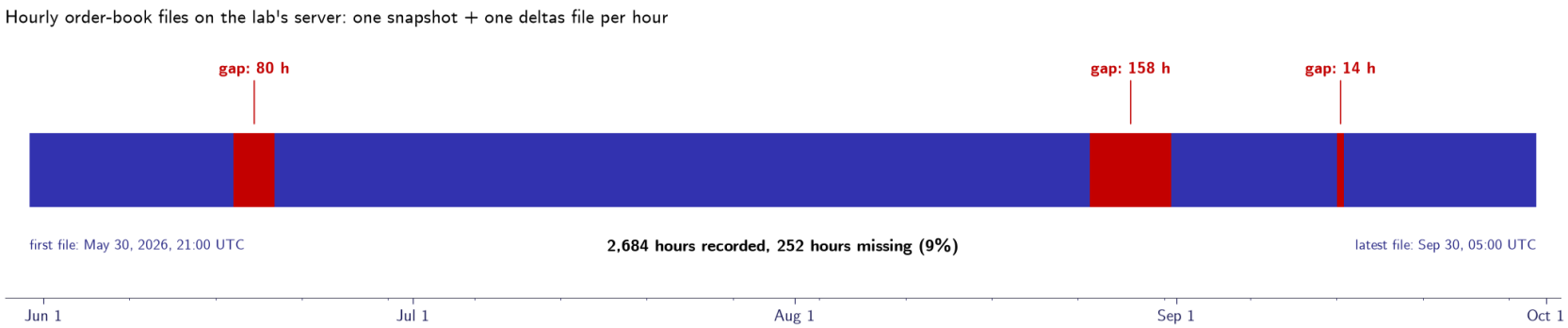
Words you will meet in every row of the data

market, `slug`	one yes/no question; the slug is its URL name and the catalog's key
event, `neg_risk`	a group of related markets; `neg_risk` = the outcomes exclude each other, so the Yes prices should sum to about $1
condition id	the market's id on the blockchain; joins catalog, order books and chain
token id, `asset_id`	the id of one share type (Yes or No); the order-book files are keyed by it
CLOB	central limit order book: Polymarket's exchange that matches orders (`clob.polymarket.com`)
maker / **taker**	rests an order / takes one; the public trade record names the taker's wallet
proxy wallet	the address Polymarket creates for each user; what the public API shows (not used in this course)
pUSD	the dollar token trades settle in, backed one-to-one by the stablecoin USDC.e
mint / **merge**	$1 in → one Yes + one No share out, and back again
snapshot / **deltas**	the whole book at the top of the hour / every level change during it
pseudonymous wallet	in the lab's trade files, a fake address that stands in for the real one; the same wallet always gets the same one
burner	a throwaway wallet made for one bet

Keep this slide open next to the notebook

Every column name in the catalog, the order-book files and the trade API is one of these twelve words.

The lab's database, part 2: the order books, hour by hour



- Two files per UTC hour, public, no login: `snap-2026-09-29T17.parquet` — the full book of every tracked market at the start of the hour `events-2026-09-29T17.parquet` — every level change during the hour (the "deltas")
- One hour of changes ≈ **1.2–1.7 million rows**, 10–14 MB, in parquet (a compressed column file). DuckDB, a Python library that runs SQL on such files, reads it straight over the web; nothing to download.

Five rows of an events file

ts_ms	asset_id	market	side	price	size
1790683200000	51356476…	0x9616c1…	BUY	0.22	4617.10
1790683200000	51989727…	0x9616c1…	SELL	0.78	4617.10
1790683200011	51356476…	0x9616c1…	BUY	0.22	4628.10
1790683200011	51989727…	0x9616c1…	SELL	0.78	4628.10
1790683200021	51356476…	0x9616c1…	SELL	0.76	2070.83

One row = "at this millisecond, on this side, the shares resting at this price became `size`" (0 = the level vanished).

2,684 hours recorded, 252 missing in three documented outages. A snapshot holds about 61,000 asset rows over about 30,800 markets. **[June's data guide; our checks, Sep 29–30 2026]**

The lab's database, part 3: every trade, with a pseudonymous wallet

- One parquet file per calendar day, public, no login: `wallet_trades/trades-2026-10-05.parquet`. **One row per fill.**
- Columns: `wallet`, `market` (the slug), `time`, `side`, `outcome`, `price`, `size`, `dollars`.
- Files reach back to mid-2025, thin at first and dense since May 30 2026; 1.1 million markets through Oct 5.
- **`wallet` is a pseudonym**: a fake address, the same one for the same wallet in every file. You can follow a wallet across markets and days; you cannot look it up anywhere.

This is the data for your final project

Group by `wallet`; join `markets` on the slug for the question, and `resolutions` for who won.

Top wallets in one market, one query (cell A8)

```
book(f"""
  SELECT wallet, round(sum(dollars)) AS volume, count(*) AS trades
  FROM '{BASE}/wallet_trades/trades-2026-10-05.parquet'
  WHERE market = 'atp-djokovi-medvede-2026-10-05'
  GROUP BY wallet ORDER BY volume DESC LIMIT 5
""")
```

wallet	volume	trades
0xd512a29e995addc51005fc9b80e7469ce7e20dce	198,159	8
0xf65a95bcd2e016e2a736adb63e8c9746018132bd	175,088	19
0xe0446ec594b827d5934e11420318d307e327337f	55,632	86
0xb73be06da617714a70c0f40b267db57d64379803	28,452	28
0x83353b9519001611c5342e471efc427627896710	27,600	1

One day of fills (Oct 3 2026): 970,571 rows, 35,355 wallets, 21,601 markets, $67.0M. Read over the web in about 2 s.

Added to the database Oct 5–7 2026. The pseudonym rule, from June's guide: each real wallet maps to one random, consistent fake address. **[June's data guide; our query, Oct 7 2026]**

Getting set up in two minutes

1. Install once

```
pip install pandas matplotlib duckdb sqlalchemy psycopg2-binary python-dotenv s
```

2. One file named .env next to the notebook

```
JUNE_DB_URL=postgresql://<the line Prof. Cheng emails you>
```

3. Two helpers, defined in the first cell

```
ask("SELECT … FROM markets …")    # the catalog  → table
book("SELECT … FROM '…parquet'")  # the order books → table
```

Wi-Fi: use eduroam.
UCLA_WEB blocks the database ports; the catalog will time out. The order-book files work on either.

Why Python

Look up a market, pull its data, compute and plot, in one place. A database GUI or plain SQL can browse the catalog, but the rest of the course lives in Python, so start there.

The notebook and the connection line go out by email after class. The line is a password: keep it out of anything public, including GitHub.
[demo/lab_database_walkthrough.ipynb]

Live: eight questions to the database

LIVE · switch to VS Code now

- ☑ **A1** What does one market look like? SELECT * ... LIMIT 1
- ☑ **A2** Find markets about a topic. title ILIKE '%bitcoin%'
- ☑ **A3** The ten most traded live markets, and a bar chart.
- ☑ **A4** One event, all its markets.
- ☑ **A5** A first statistic: the histogram of live prices.
- ☑ **A6** Save a table to CSV in one line.
- ☑ **A7** Peek into one hour of order-book data.
- ☑ **A8** One market's trades, grouped by wallet.

Follow along

Each cell is one question and one table back. Change the word inside the quotes in A2 and run it again: that is already your own query.

If the Wi-Fi fails

The same eight cells, with their outputs, are the slides labelled **Backup 1–8** at the end of this deck.

Part B: rebuild an order book. Which market was busiest?

Code

```
HOUR = "2026-09-29T17"  # Tue, 10 am in LA

busiest = duckdb.sql(f"""
    SELECT market AS condition_id,
           count(*) AS level_changes
    FROM '{BASE}/events-{HOUR}.parquet'
    GROUP BY market
    ORDER BY level_changes DESC LIMIT 5
""").df()
# then join the catalog for the titles
```

Result (0.2 s)

market	level changes	24 h volume
Musk <40 tweets, Sep 28–30	**55,160**	$38,526
Andersson next PM of Sweden	52,276	$213
Democrats win Maine Senate	44,348	$1,042
Parti Québécois most seats	43,928	$179
Republicans win Maine Senate	39,044	$720

- Two sources joined on `condition_id`: the parquet file gives the counts, the catalog gives the names.
- A busy book is not a busy market: four of the five traded under $1,100 that day. Bots re-quote constantly.

Output of the notebook, Part B, steps 2–3. Full market titles in the notebook. **[our database, hour 2026-09-29 17:00 UTC]**

Part B: trust, but verify

Code

```
# seed the book with the snapshot, replay every change
path, bids, asks = replay(seed_bids, seed_asks, deltas)

# a real book never has bid ≥ ask. Count violations.
crossed = int((path.spread <= 0).sum())
stale = crossed > 0.01 * len(path)
```

The docs said "snapshot, then replay".
The data said: 3 of 8 snapshots are stale.

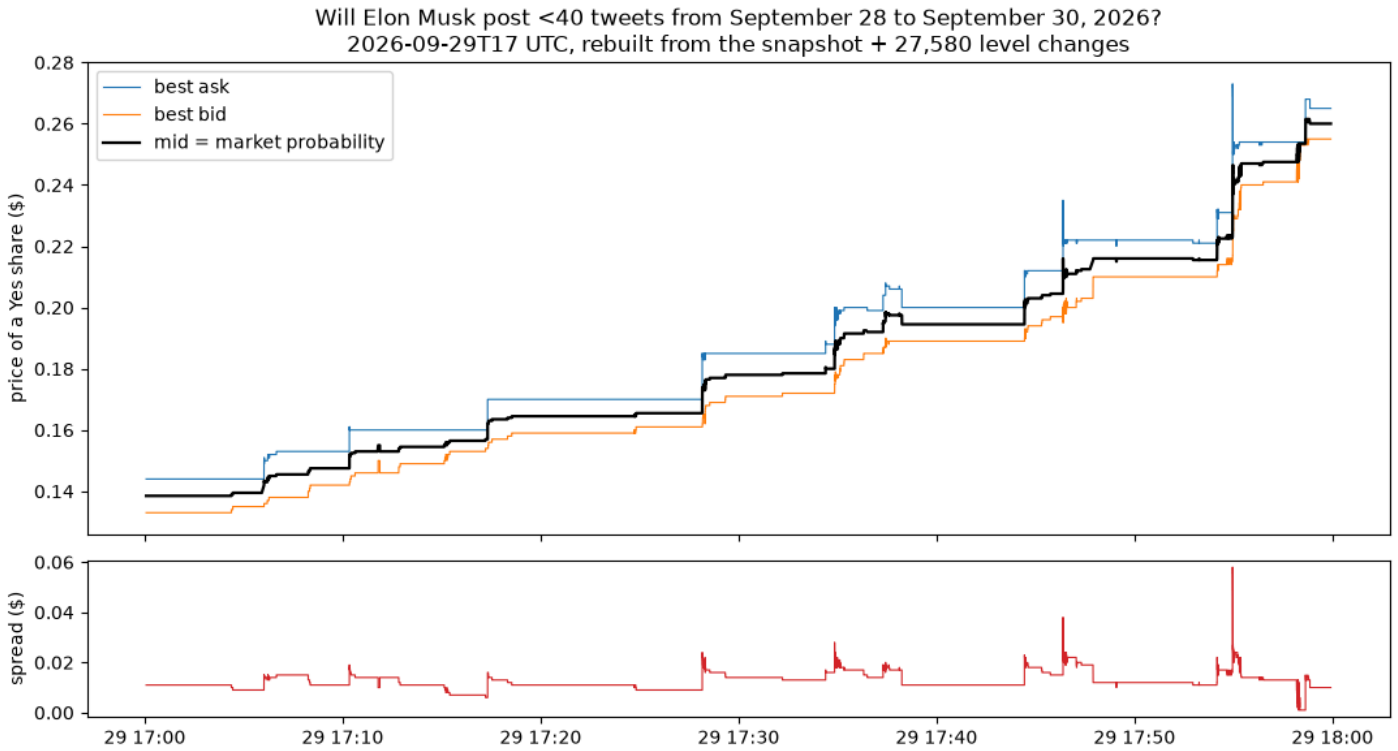
The recorder kept re-sending an old book for those markets (the Swedish one *ended Sep 13*). Replaying from a stale snapshot gives an impossible book, bid above ask, all hour. The notebook detects it and rebuilds from the deltas alone instead.

The 8 busiest markets of the hour

market	snapshot	crossed	
Musk <40 tweets (ours)	.133 / .144	0	OK
Andersson next PM of Sweden	.660 / .670	26,126	STALE
Democrats win Maine Senate	.590 / .600	0	OK
Parti Québécois most seats	.930 / .950	0	OK
Republicans win Maine Senate	.390 / .400	0	OK
Siegmund Minister-President	.380 / .390	16,970	STALE
Eizenkot next PM of Israel	.523 / .528	16,320	STALE
Musk 160–179 tweets	.167 / .170	1	OK

Notebook Part B, steps 5 and 7. A crossed moment is one level change after which best bid ≥ best ask; one is timing noise, thousands mean a stale snapshot. **[our database, hour 2026-09-29 17:00 UTC]**

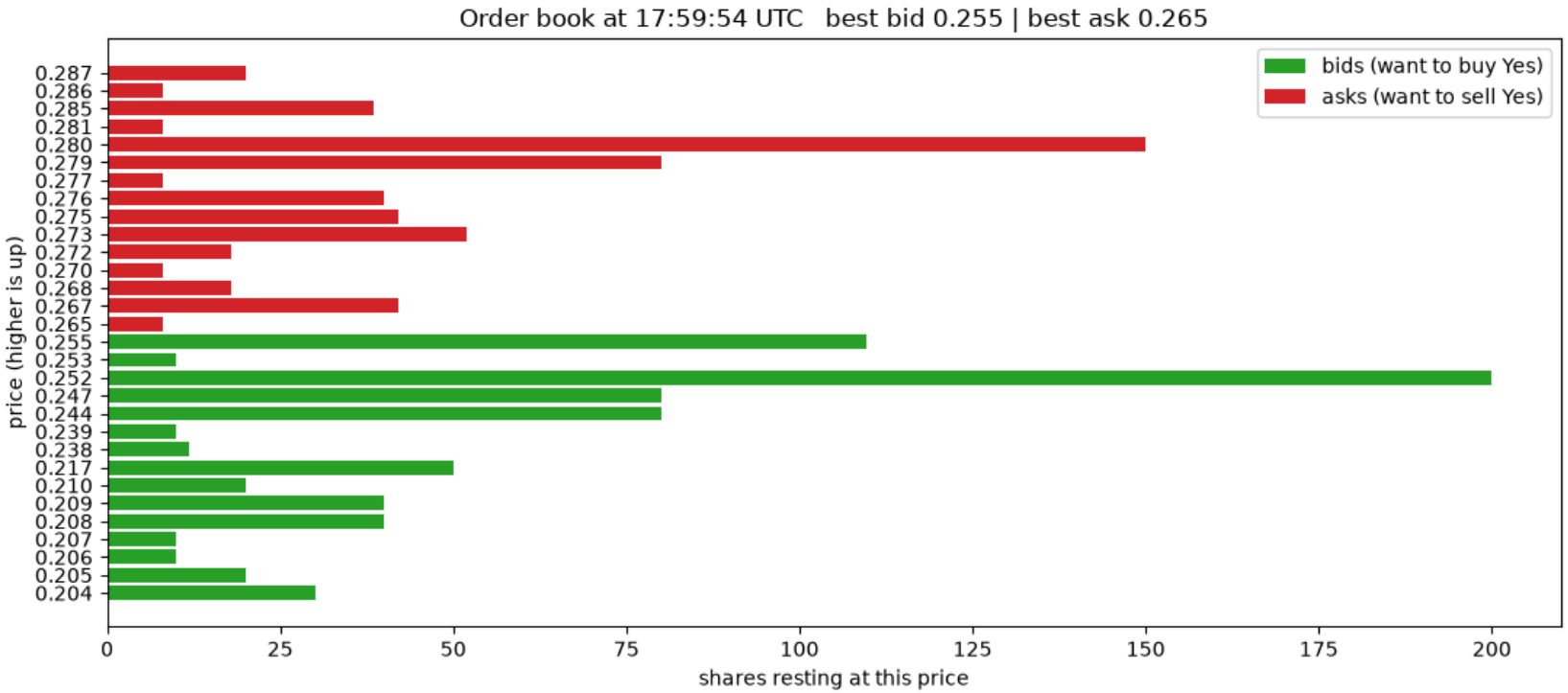
Part B: the book rebuilt tick by tick



- Every step is one order arriving or leaving. The **mid** is the market's probability, second by second: from 14% to 26% in one hour.
- The **spread** (bottom) widens for a moment at every jump: a taker ate the best level, and it took a few seconds for a maker to replace it. Slide 10, seen in the data.

27,580 level changes on the Yes side replayed from the 17:00 snapshot (slide 29's 55,160 counted Yes and No). Full code in notebook Part B, steps 4–5. **[our database, hour 2026-09-29 17:00 UTC]**

Part B: the book at the end of the hour



- The same picture as slide 9, but **reconstructed** from the recording for a moment in the past, 17:59:54 UTC.
- This is a thin book: most levels hold 10–50 shares at about 25¢ each, the biggest 200. The whole visible book is worth a few hundred dollars.

Fifteen best levels on each side, from the replayed book. Notebook Part B, step 6. **[our database, hour 2026-09-29 17:00 UTC]**

Quirk 1: the status flags are never updated

1,178,126
markets in the catalog

0
flagged `closed`, `archived` or inactive

1,023,839
have an `end_date` in the past (87%)

Looks right, returns everything

```
SELECT * FROM markets WHERE active AND NOT closed
```

Use the date instead

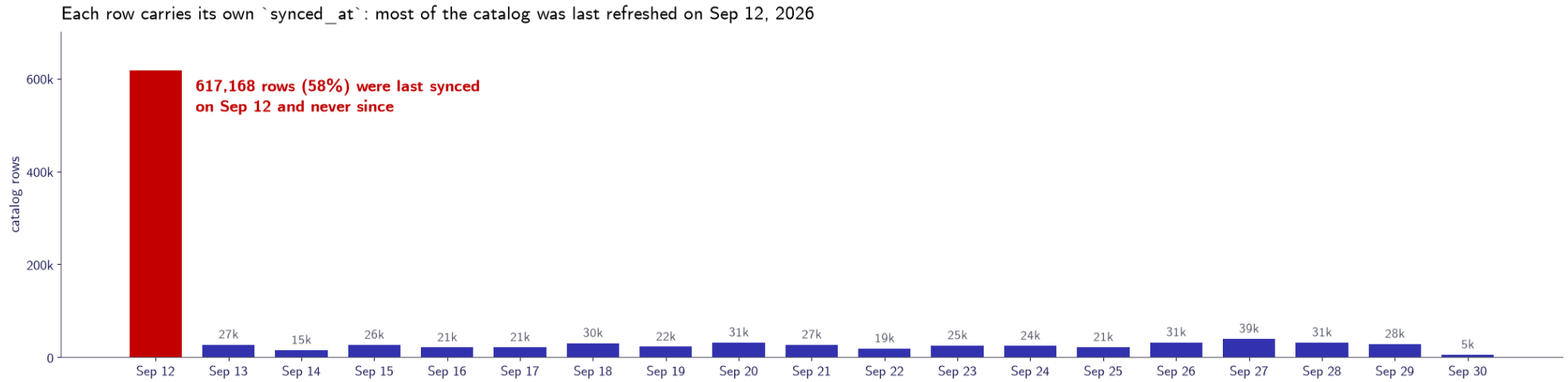
```
SELECT * FROM markets WHERE end_date > now() OR end_date IS NULL
```

Why it happens, and two caveats

The flags were copied once, when a market was first seen, and never refreshed; dates are set at creation and never need refreshing. Caveat 1: about 3,000 markets have no end date (open-ended races); keep them with `OR end_date IS NULL`. Caveat 2: **`end_date` is the scheduled close, not the actual one**. Of 213,602 resolved markets, 78,424 closed more than an hour early and 6,708 more than a day late (Ethiopia's next prime minister: scheduled Jun 1, resolved Oct 6). For the real close and the winner, join `resolutions`.

Counts as of Oct 7 2026. The documentation's own example query for "active markets" filters on the flags. **[our database]**

Quirk 2: every row has its own clock



- `last_trade_price`, `best_bid`, `volume_24hr` are **as of that row's `synced_at`**, not as of now.
- The Bitcoin market's row says 34¢; the live API said 49.5¢. Its row is from Sep 12, and so is the median row. Measure it before you trust it:

```
ask("SELECT to_timestamp(percentile_cont(.5) WITHIN GROUP (ORDER BY synced_at))"
    "  AS median_sync, to_timestamp(max(synced_at)) AS newest FROM markets")
→ median_sync 2026-09-12 21:24   newest 2026-10-07 22:17
```

Rule

Use the catalog for what **never changes**: title, dates, ids, event. For anything that **moves** (price, volume, status), ask the live API, the order-book files, or `resolutions`.

Median `synced_at` is Sep 12 21:24 UTC; the newest is Oct 7 22:17. Only a few tens of thousands of rows are refreshed per day. **[our database, Oct 7 2026; gamma-api.polymarket.com for the live price, Sep 30]**

Quirk 3: which markets are being recorded?

The 7,998 markets with ≥ \$10,000 of daily volume — are they in the snapshot?



The 30,793 markets in the snapshot — how big are they?



- If the rule were "≥ \$10k daily volume is recorded", every big market would be recorded, and only big ones. Instead **29% of the big markets are missing**, and **82% of the recorded markets are small or already ended**.
- And "daily volume" here is the stale `volume_24hr` of Quirk 2, so part of the 29% may be staleness, not missing coverage. Either way: **check that your market is in the snapshot** before you build on it.

Flagged, not settled

We have asked June, who runs the recorder, what the rule is. Until then, order-book coverage is **unknown**; the trade files and the catalog are not affected.

Snapshot of 2026-09-30 05:00 UTC joined to the catalog on `condition_id`. **[our database]**

What this database can and cannot tell you

It can

- List every market and event, with rules, dates, ids and **who won**.
- Give **every trade** with a **pseudonymous wallet**: group by wallet, follow it across markets.
- Give the **order book at any second** of any recorded hour since May 30.

It cannot

- Tell you **who a wallet is**. The address is a pseudonym by design; that is the rule of this course.
- Tell you the **current** price or status (quirks 1 and 2).
- Promise complete order-book coverage (quirk 3), or fill the three recording gaps.

The final project runs on this database

Wallet trades, who won, and the order books: everything Block 3 needs is here. Polymarket's public API (slide 15) shows real addresses; in this course we do not use them.

`resolutions` and the `wallet_trades` files were added Oct 5–7 2026; `market_hourly` is still empty. The notebook's Part B step 8 shows the new tables. **[our database; June's data guide]**

From a category to its wallets, inside the database

```
# 1. a category -> its markets  (tags: politics, tech, pop-culture, business, economy)
ev = get("https://gamma-api.polymarket.com/events",
         params={"tag_slug": "tech", "closed": "false", "limit": 20, "order": "volume"})
slugs = [m["slug"] for e in ev for m in e["markets"]]   # the slug: key into our database

# 2. their trades, from the lab's day files (one month, listed)
days = [f"'{BASE}/wallet_trades/trades-2026-09-{d:02d}.parquet'" for d in range(1, 31)]
tr = book(f"""
    SELECT wallet, market, time, side, outcome, price, dollars
    FROM read_parquet([{", ".join(days)}])
    WHERE market IN ({", ".join(repr(s) for s in slugs)})
""")

# 3. one row per wallet
W = tr.groupby("wallet").agg(spent=("dollars", "sum"), trades=("dollars", "size"),
                             markets=("market", "nunique"))
```

```
tag "tech" → 20 events → their slugs → September's 30 day files → one row per wallet.
Who won: ask("SELECT market_slug, resolved_yes FROM resolutions WHERE ...")
```

- Three keys: the **slug** joins catalog, trades and resolutions; the **condition id** and **token id** join the order books.
- Price history: `clob…/prices-history?market=<tokenId>&interval=max`, public.
- Outside the course, Polymarket's public trades API returns the same trades with **real** addresses. We do not use it here.

If your team picks Kalshi

`api.elections.kalshi.com/trade-api/v2`: `/markets`, `/markets/trades?ticker=…`, `/series/…/candlesticks`. Public, no key, no trader identities. Fine for the midterm; for the final, a Kalshi team works at the market level: unusual price or volume moves before the news.

Gamma tag call tested Sep 30 2026 (`data_snapshots/student_recipe_tech.json`); the day files tested Oct 7 2026. `ask` and `book` are the notebook's two helpers. **[gamma-api.polymarket.com; our database; api.elections.kalshi.com]**

Validate what you collect

- [ ] **Count before you trust.** A filter that returns everything, or nothing, is wrong. (the status flags)
- [ ] **Ask "as of when?"** for every number. Read the sync or timestamp column. (34¢ vs 49.5¢)
- [ ] **Test an invariant.** Bid $<$ ask. Yes $+$ No $=$ \$1. A market ends after it starts. Count the violations. (three stale snapshots)
- [ ] **Cross-check two sources** on the same fact: catalog vs live API, snapshot vs deltas, our data vs the chain.
- [ ] **Check coverage**: is your market, your period, even in the data? (three gaps, 29% of big markets)
- [ ] **Write down what you found** and tell the owner. Every dataset has a maintainer who wants to know.

Why this is the real lesson of Block 2

Everything on this list came out of a two-hour first look at this database. None of it is in the documentation. An analysis that did this is worth more than an exciting result on unchecked data.

The lab found its own version of this in August: two components of the recorder disagreed on a volume threshold (\$100 vs \$10,000), and the mismatch was caught only by counting what each one saw. **[lab notes, Aug 2026]**

What to take from Block 2

1. Polymarket data has **three layers**: catalog, trades and prices, the chain. Slug, `condition_id`, wallet join them.
2. The **order book vanishes** unless someone records it. The lab has, hourly, since May 30.
3. The database is **SQL for the catalog, DuckDB for the books**, both through two helper functions. Use **eduroam**.
4. Three quirks: filter on **end_date**, read **synced_at**, check that your market is **in the snapshot**.
5. **Every trade with a pseudonymous wallet**, and **who won**. Never try to identify a wallet.
6. **Validate what you collect.** Count, ask as-of-when, test an invariant, cross-check.

After the break (5:30)

Using the data: confirmed insiders, a first detector on real wallets, open questions.

Block 3: using the data to find insiders

1. **The confirmed cases**: who got caught, how much, and how it came out.
2. **What an insider looks like** in the trade data: one row per wallet.
3. **A first detector**: Isolation Forest, a preview of Week 5, run live on 55,952 real wallets.
4. **Evaluating with a handful of labels**, against chance and against a simple baseline.
5. **The wallets no detector sees**, and why **related markets** matter.
6. **What you could ask of this data**: six open questions.

By 6:20 you should be able to

turn trades into a wallet table, rank wallets by how unusual they are, judge that ranking honestly with few labels, and say what a per-wallet model cannot catch.

What "insider" means here, and how a case becomes confirmed

An **insider**: someone who bet on an outcome they knew in advance. We observe the bet, never the knowledge, so "confirmed" has two grades:

1 **Charged**	A prosecutor or regulator named a person: a US Army soldier (Maduro market), a Google engineer (Google search markets).	`2 cases`
2 **Reported**	Analysts linked fresh wallets to a leak in public (Bubblemaps, Forbes, Decrypt, CNN, an SSRN paper). Wallets named; people usually not.	`the rest of the list`

The labels are few and biased

39 rows, 36 distinct wallets: the insiders *loud enough to get caught*. Every quiet one is missing. You work with **pseudonymous ids**; a label marks a wallet, never a person.

Public sources: SSRN 6426778 (2026); Bubblemaps and Forbes (Mar 2, May 11 2026); Decrypt; CNN (Mar 24 2026); Sidley Austin on the Maduro case. **[lab case list, 2026-07]**

The confirmed cases, grouped by event

Event	Wallets	Profit	How it came out
US strikes Iran · Feb 2026	9 accounts	$2.4M	Fresh wallets, funded days before; Bubblemaps, Forbes
US × Iran ceasefire · Apr 2026	9 accounts	up to $600k	Fresh wallets again, six weeks later; Bubblemaps
Maduro out · Jan 2026	1 account	$404k	**Charged**: a US Army soldier
Machado, Nobel Prize · Oct 2025	3 accounts	$87k · $72k · $31k	SSRN paper
Google search, Gemini · 2025	1 account	$0.9–1.2M	**Charged**: a Google engineer
OpenAI browser · Oct 2025	5 accounts	$28k (one wallet)	SSRN paper
Swift–Kelce engagement · Aug 2025	1 account	$3.1k	SSRN paper
5-minute Bitcoin market · Apr 2026	1 account	$20k	Bubblemaps

- Politics, technology, entertainment, economy: **your four project categories**; the first two dominate the confirmed list.

Profits as reported in the public sources, not recomputed. One odd case omitted: a Paris temperature market. Wallets are counted, not named: in the course every wallet is a pseudonymous id. **[public reports; lab case list, 2026-07]**

What an insider looks like in the trades: one row per wallet

Two example features	The question it asks
`wallet_age_days`	First to last trade. **Fresh wallets are free**: a wallet made for one bet has no history to be out of character with.
`concentration`	Share of all spending in one market. **All-in?** Someone who knows one thing bets on one thing.
and four more	How big the bets were, relative to the wallet and to the market; what price was paid; how many markets. All in the table you receive.

Why one row per wallet, not per trade?

A trade is an event. A wallet is a **behaviour**. "Out of character" needs a character, so every feature compares a wallet to its own history and to the market it traded in.

From Block 2 to here

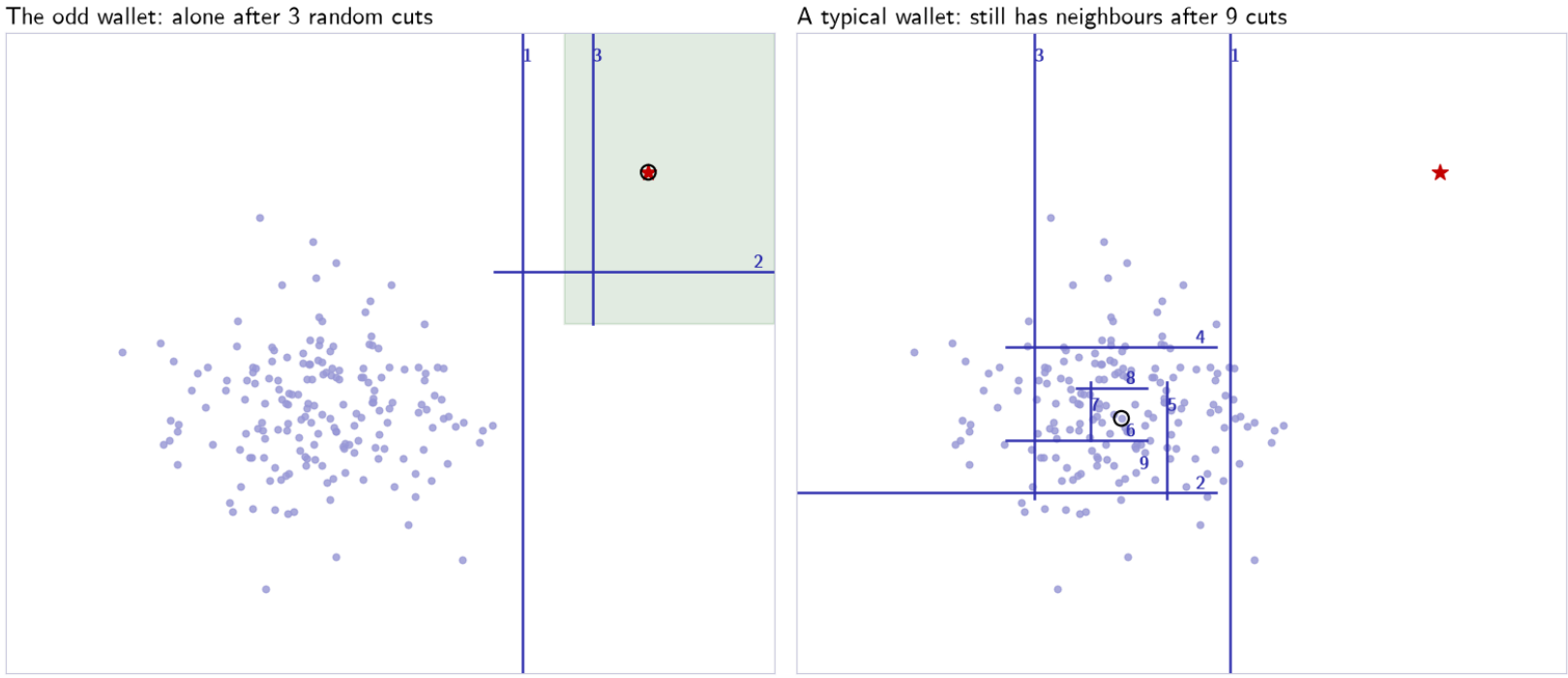
Every feature is a `groupby("wallet")` over the trade rows you saw on slide 22. Nothing here needs more than pandas.

Features are a choice

These are one reasonable set, not the right one. Never use a feature that needs the **outcome** of the market: a model that knows who won has nothing left to detect.

Features computed from the trade rows by `demo/_build_wallet_table.py`; each column is defined in the notebook's first cell. **[demo/insider_demo.ipynb]**

Preview of Week 5: isolation



Each cut picks a random feature and a random threshold inside the current cell. Short path to isolation = anomalous. Illustration on synthetic points.

- Pick a random feature and a random threshold; cut. Repeat inside the cell that holds the point until it is alone.
- An unusual point is alone after **few** cuts. A typical point, deep in the crowd, needs **many**.
- No labels, no model of "normal". Just: **how hard is it to separate you from everyone else?**

Illustration on synthetic points. The method: Liu, Ting & Zhou, "Isolation Forest" (2008). Prof. Cheng covers it properly, with the other tree methods, on Oct 22 (Week 5). **[drawn for this lecture]**

The score, and why the bots come first

- $h(x)$ = number of cuts to isolate x in one random tree. Average over 200 trees: $E[h(x)]$.
- Normalise by $c(n)$, the average depth of a random point among n:

$$s(x) = 2^{-E[h(x)] \,/\, c(n)}$$

- s near **1**: isolated fast, anomalous. Near **0.5**: ordinary. We keep the **ranking**, not a threshold.
- Each tree sees a random sample of 256 wallets, so a forest on 56,000 wallets fits in **seconds**.

Stage 1: set the bots aside

Market-making bots trade thousands of times across hundreds of markets and are the most anomalous wallets by any measure. Left in, they fill the top of the ranking and the insiders are buried under them. Rule for today: **≥ 500 markets or ≥ 2,000 trades**.

Stage 2

Isolation Forest on the remaining human wallets, the features of the wallet table, 200 trees. On today's table the bot effect is mild (about a quarter of an unfiltered top 1% would be bots); we keep the two stages because bots are a confounder in *every* population.

Market makers and bots are the first confounder of any wallet-level analysis. One confirmed insider (3,821 trades) is itself caught by the bot rule and never scored: rules have costs. **[demo notebook, section 2]**

Behind the demo: from 8.4 million trade rows to one row per wallet

The recipe (pandas)

```
# every row: who bought? taker on BUY rows, maker on SELL rows
tr["buyer"] = where(tr.direction == "BUY", tr.taker, tr.maker)

# spend per wallet and market, over the whole history
wm = tr.groupby(["buyer", "market_id"]).agg(
        spend=("usd_amount", "sum"), max_single=("usd_amount",
        first=("timestamp", "min"), last=("timestamp", "max"))

# one row per wallet
W = wm.groupby("buyer").agg(...)      # full recipe: _build_wa
W["concentration"] = wm.groupby("buyer").spend.max() / W.total
```

The result: `data/wallet_features.csv`

55,952 wallets, 19 of them publicly confirmed insiders

wallet	pos_ratio	mkt_share	price	conc	age_d	mkts	confirmed
w_b6640fc6a7	0.365	0.000322	0.576	1.000	1.60	1	False
w_ce85723407	1.502	0.000362	0.937	0.406	49.93	4	False
w_b92e826e83	1.100	0.000021	0.341	0.483	17.90	4	False
w_ab04538147	2.332	0.000559	0.204	0.389	19.12	6	False
w_09a7eb8ca6	1.009	0.000114	0.679	0.078	4.87	13	False

- Population: every wallet that bought in the **five event groups** of the cases table (Iran strikes, ceasefire, Machado, Maduro, OpenAI), at least 3 buys.
- Features over the wallet's **whole history** in the lab's files: 1,644 markets, Apr 2025–Mar 2026. Both sides of each trade count, as in the lab's day files (slide 27).

The 1.8 GB trade file is not distributed; the 10 MB wallet table is, with the notebook and the build script. Each transaction also carries a row that restates the taker's whole order; the script drops it, or the taker is counted twice. **[demo/_build_wallet_table.py; demo/insider_demo.ipynb]**

Live: fit the forest, read the top 1%

LIVE · switch to VS Code now

- [x] **1** Load the table: 55,952 wallets, 19 labels.
- [x] **2** Stage 1: 284 bots set aside, 55,668 wallets to score.
- [x] **3** Stage 2: `IsolationForest(n_estimators=200, contamination=0.05, random_state=42).fit(X)` — a few seconds.
- [x] **4** Sort by score. Who is in the top 1%?

If the notebook fails

The slides labelled **Backup 9–12** at the end of the deck hold the same cells with their outputs.

What comes back (section 4)

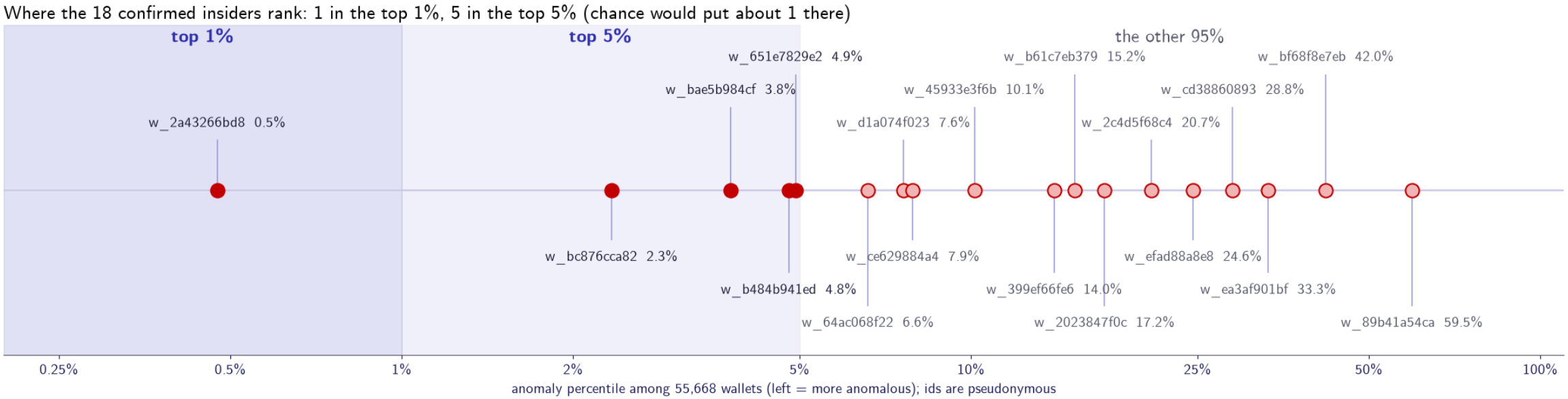
```
top 1% = 557 wallets. Confirmed insiders among them: 1

wallet          score  insider  pos_ratio  mkt_share  age_d  markets
w_3c83e74381    92.0             19.2       0.18       329    224
w_592a48ad5c    91.8             23.1       0.22       330    186
w_4fad99896f    91.8             23.6       0.17       347    264
w_a085741c51    91.6             36.3       0.29       172    233
w_8577bb2d2f    91.1             30.8       0.18       274    135
w_a8a52f1188    91.0             21.7       0.09       279    221
…
```

- The forest's top is **heavy, long-lived traders**: median 88 markets and 260 days in the top 1%, against 4 markets and 21 days in the population. Semi-professionals, not burners (throwaway wallets made for one bet).

Outputs of the notebook run on Sep 30 2026; the seed is fixed, so students reproduce the same ranking. **[demo/insider_demo.ipynb, sections 1–4]**

Where the 18 confirmed insiders landed



- Flag the top 5% (2,784 wallets) and you catch **5 of the 18** known insiders. Chance would give you about **1**: a lift of 5.6.
- The other **13** sit among the ordinary. Not because they are subtle: mostly because of what they *are* (slide 53).

Percentile = rank ÷ 55,668 scored wallets. One further confirmed insider (3,821 trades) was set aside by the bot rule and is not shown. **[demo/insider_demo.ipynb, section 5]**

The one hit, next to the middle of the ranking

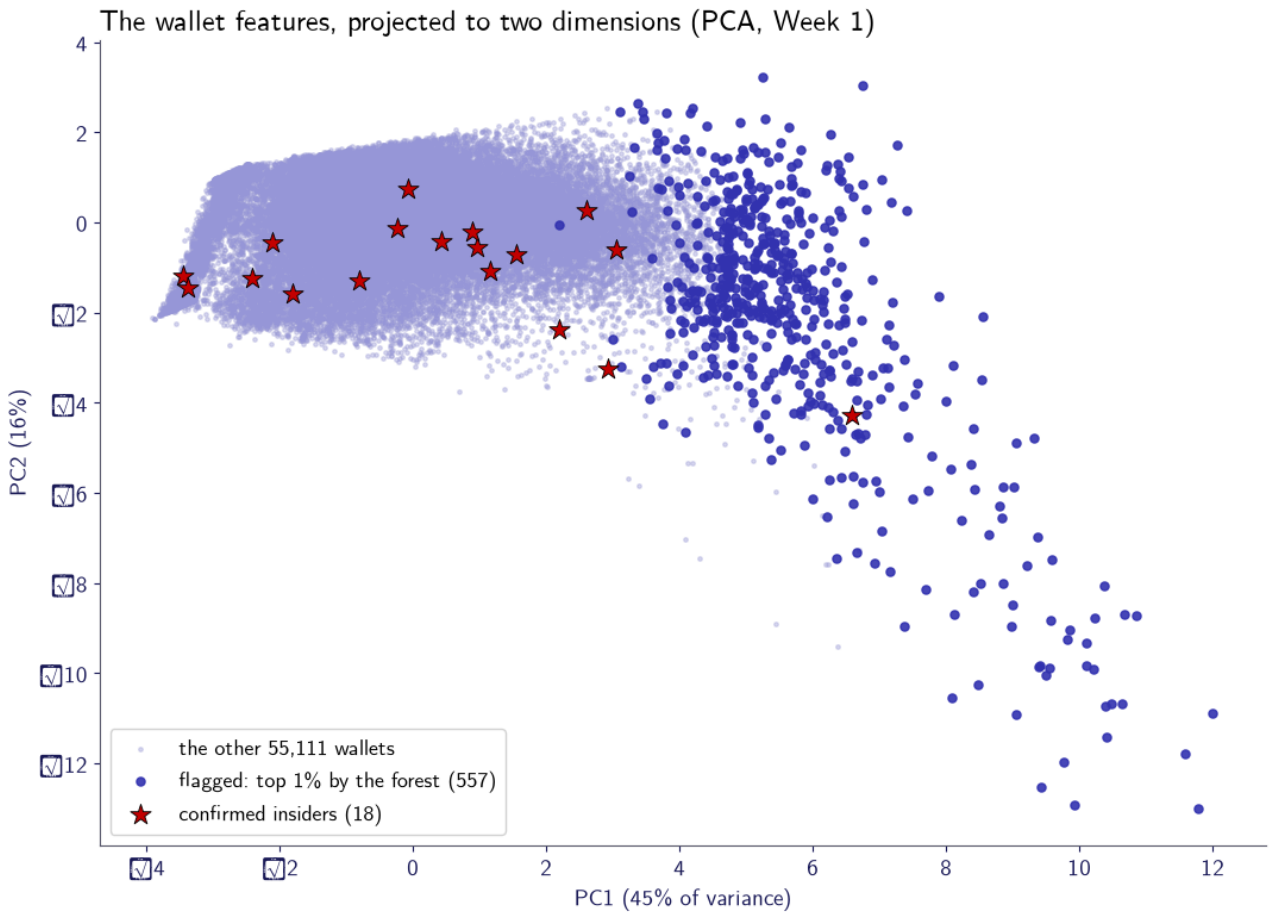
feature	w_2a43266bd8 (confirmed insider, Machado Nobel market)	w_8379b038ff (rank 27,787 of 55,668: the middle)
wallet_age_days	301	19
concentration	0.15	0.18
unique_markets	55	12
largest share of one market's volume	**0.261** (a quarter of the market)	0.0004
trades · total spent	1,332 · **$675,487**	23 · $306
forest rank (1 = most anomalous)	**264** (top 0.47%)	27,787

Not the stereotype

No fresh burner: 55 markets, 300 days, two-thirds of a million dollars. What gives this wallet away is **size relative to the market**. The stereotype (fresh, all-in, one market) is one kind of insider, not the kind.

Both wallets from the demo population (55,952 wallets active in the five event groups; features over their whole history, 1,644 markets). The "ordinary" wallet is a real wallet from the middle of the ranking. Ids are pseudonymous. **[demo/insider_demo.ipynb, section 7]**

The whole population in two dimensions



- Week 1's PCA on the (log, standardised) wallet features. The first axis is **size and history**: many markets, long life, big money.
- The forest's top 1% (blue) sits at the far end of that axis: the heavy traders.
- The confirmed insiders (stars) are **spread through the crowd**. Only a few sit near the blue group.

What the picture says

"Unusual" and "insider" are different things. The forest finds the unusual. Some insiders are unusual. Most of the caught ones were not, in these numbers.

Variance explained by the two components is printed on the axes. **[demo/insider_demo.ipynb, section 8]**

Evaluating with a handful of labels

flag the top	wallets	forest	biggest spenders first	chance
0.5%	279	1 of 18	4	0.09
1%	557	1	5	0.18
2.5%	1,392	2	9	0.45
5%	2,784	**5**	**13**	0.90
10%	5,567	8	16	1.80

Confirmed insiders caught, by ranking. The simple baseline wins.

1. **Ranks, not probabilities.** The score is not P(insider). Report "how far down the list" and "how many of the known ones are above the line".
2. **Always run a simple baseline.** Here "biggest spenders first" beats the forest, and that is the finding: the *confirmed* insiders are the ones the press caught, and the press caught the biggest winners. Size finds them. The forest ranks by how unusual a wallet is, a different question.
3. **Recall against the labels is an upper bound on the conspicuous kind.** A wallet without a label is not a confirmed non-insider: the labels are a biased sample of the positives, and the unlabeled rows are a mixture of both.

Chance = 18 × k. **[demo/insider_demo.ipynb, section 6]**

The burner problem: the insiders the forest cannot see

missed insider	markets	age (days)	all-in	spent
w_2023847f0c	1	0.02	1.00	$18,136
w_efad88a8e8	1	0.00	1.00	$1,865
w_2c4d5f68c4	2	0.05	0.75	$14,392
w_cd38860893	2	2.25	0.71	$75,327
w_399ef66fe6	4	8.3	0.80	$16,210
w_bf68f8e7eb	9	5.7	0.20	$46,029

Six of the thirteen misses (pseudonymous ids). All Iran-strike wallets, all a few days old.

- A wallet created yesterday, one market, a burst of bets, gone. In these features that is **not anomalous**: thousands of one-shot gamblers look exactly like it.
- "Out of character" needs a character. A burner has none. Size does not help either: these wallets spent $2k–$75k, not millions.

The limit of per-wallet features

Nothing in *how* a burner bets separates a burner insider from a burner gambler. What could is **context the single wallet does not carry**: the timing against the news, and what moved in the related markets.

The misses table is section 9 of the notebook; the other seven misses are older wallets with small positions. Median wallet in the population: 4 markets, 21 days, $196. **[demo/insider_demo.ipynb]**

Why related markets matter

1	**Market A**	"...by March 1?" One wallet: a modest position, a few percent of the book.
2	**Market B**	"...by March 15?" Same wallet, same side, another modest position.
3	**Market C**	"...by March 31?" And again.
Σ	**The event**	One piece of information, one bet, spread thin over a family of markets. Large only when you add it up.

- Slide 18: one piece of news moved **30 Bitcoin markets** at once. An **event** is a family of markets, and whoever knows something about X knows it for every market about X.
- **Observed:** the Iran-strike wallets held positions across the event's date markets. Each looked modest; the sum was large. Public reports read this as one operator.
- So "how much did this wallet bet?" is a question about a **group** of markets. Any per-market screen can be split around.

Which markets belong together?

Same event, same tags, prices that move together, the same word in the question. How to group them is a modelling choice, and it is **yours**.

Schematic on the left. Iran spreading: Bubblemaps and Forbes on the Feb 28 2026 market; the lab's files hold 133 markets in that event group. The catalog's `event` field (slide 25) is the grouping you get for free. **[public reports; our database]**

What you could ask of this data

Pattern	What you would look for	Where
Big bets right before big news	A wallet buys heavily hours before an announcement, at a price far from where the market ends up.	wallet trades (slide 27)
Price moves ahead of the headline · either platform	Flat at 30¢ for a week, then 70¢ in two hours, and the story explaining it appears afterwards. Few large orders or many small? Was any public source out when the move began?	price history, order books (slides 8, 9)
Fresh wallets with one perfect bet	First trade ever, all-in on one market, never trades again. How many such wallets are there, and how many of them won?	wallet table (slide 47)
The same information across sibling markets	Many small positions across one event's markets that add up to a large one.	catalog + wallet trades (slide 54)
Who is normal?	Bots and market makers trade constantly and dominate volume. What does an ordinary participant look like, and how do you tell the two apart?	order books + trades
Lucky, informed, or inside?	A long record of winning bets may be domain knowledge. What evidence in the data would separate the three?	all of the above

Whatever you find in this data that we have not, is your finding

Rows 1, 3–6 need the wallet in each trade (Polymarket). Row 2 needs only prices and volume: Polymarket or Kalshi.

What to take from today

1. A market is a question with a **\$1-or-\$0** answer; its price is a crowd's probability, made by an **order book**.
2. Polymarket publishes **every trade with a wallet**. The lab's database adds the one thing the API forgets: the **book over time**. Use eduroam.
3. **Validate what you collect**: count, ask as-of-when, test an invariant.
4. Insiders leave marks in a **wallet table**. A forest finds the *unusual*; the confirmed insiders were mostly just *big*.
5. Evaluate with **ranks, precision at k, chance and a baseline**; the labels are the caught ones.
6. Burners beat per-wallet features. Information moves whole **families of markets**.

By email: both notebooks, the pseudonymous wallet table, the connection line

Whatever you find in this data that we have not, is your finding. Do not try to identify anyone behind a wallet.

Backup A1: what does one market look like?

```
ask("""
    SELECT * FROM markets
    WHERE end_date > now()
    ORDER BY volume_1wk DESC NULLS LAST
    LIMIT 1
""").T
```

.T turns the row on its side so every column is readable.

slug	us-x-iran-permanent-peace-deal-by-december-31-2026-961-5…
title	US x Iran permanent peace deal by December 31, 2026?
condition_id	0x9769f78cbc95a5ed11895e6064bac471d8fd8f930b260cf581b68d…
synced_at	1789248242
start_date	2026-05-04 14:49:25 UTC
end_date	2026-12-31 00:00:00 UTC
active	True
closed	False
neg_risk	False
enable_order_book	True
last_trade_price	0.973
best_bid	0.972
best_ask	0.973
volume_clob	11655904.39456
volume_24hr	1006166.303627
volume_1wk	4657350.468455
token_ids	[104718292808309433767402444061650910790437726449955521 7…
outcomes	[Yes, No]
event_slug	us-x-iran-permanent-peace-deal-by

Backup A2: find markets about a topic

```
ask("""
    SELECT title, last_trade_price, volume_1wk, end_date
    FROM markets
    WHERE title ILIKE '%bitcoin%' AND end_date > now()
    ORDER BY volume_1wk DESC NULLS LAST
    LIMIT 10
""")
```

	title	last_trade_price	volume_1wk	end_date
0	MicroStrategy sells any Bitcoin by December 31, 2026?	0.997	679612.899956	2027-01-01 05:00:00+00:00
1	Will Bitcoin reach $92,500 in September?	0.005	437861.318931	2026-10-01 04:00:00+00:00
2	Will Bitcoin dip to $45,000 by December 31, 2026?	0.080	376379.841026	2027-01-01 05:00:00+00:00
3	Will Bitcoin dip to $60,000 by December 31, 2026?	0.280	255041.986607	2027-01-01 05:00:00+00:00
4	Will Bitcoin reach $95,000 by December 31, 2026?	0.340	170995.616308	2027-01-01 05:00:00+00:00
5	Will Bitcoin dip to $45,000 by December 31, 2026?	0.380	154217.132307	2027-01-01 05:00:00+00:00
6	Will Bitcoin dip to $40,000 by December 31, 2026?	0.060	151359.828550	2027-01-01 05:00:00+00:00
7	Will Bitcoin dip to $35,000 by December 31, 2026?	0.050	133170.090168	2027-01-01 05:00:00+00:00
8	Will Bitcoin dip to $55,000 by December 31, 2026?	0.530	129540.990946	2027-01-01 05:00:00+00:00
9	Will Bitcoin reach $1,000,000 by December 31, 2026?	0.009	123726.419142	2027-01-01 05:00:00+00:00

Change `bitcoin` to any word. `ILIKE` ignores case; `%` means "anything".

Backup A3: the ten most traded live markets

```
top = ask("""
    SELECT title, last_trade_price, volume_1wk
    FROM markets
    WHERE end_date > now()
    ORDER BY volume_1wk DESC NULLS LAST
    LIMIT 10
""")
top.set_index("title").volume_1wk.astype(float).div(1e6
   .sort_values().plot.barh()      # numbers come back
```



Backup A4: one event, many markets

```
event = ask("SELECT event_slug FROM markets WHERE end_date > now() ORDER BY volume_1wk DESC NULLS LAST LIMIT 1").event_slug[0]
ask("SELECT title, end_date, last_trade_price, volume_1wk FROM markets WHERE event_slug = :e ORDER BY end_date", e=event)
```

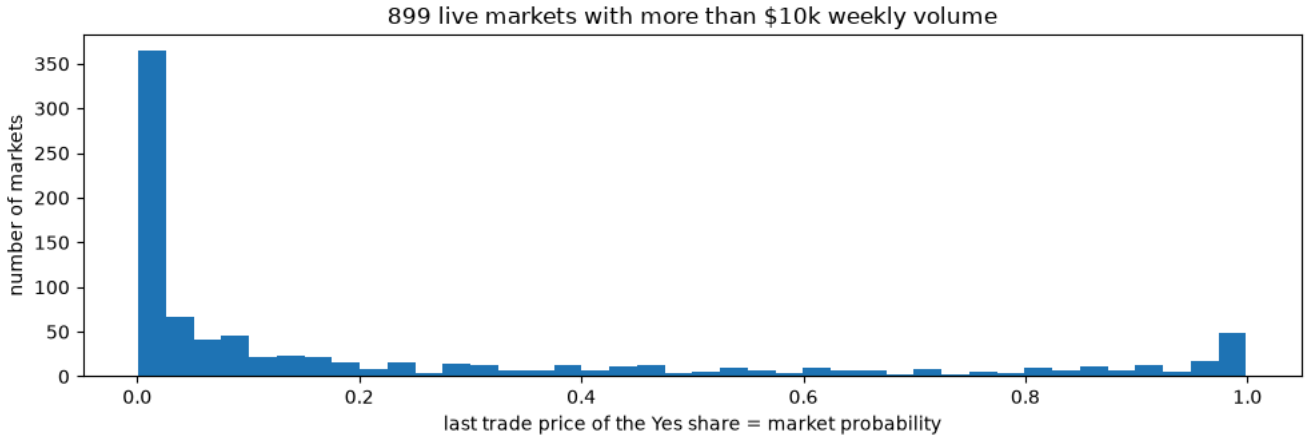
```
event: us-x-iran-permanent-peace-deal-by
                                                title                  end_date  last_trade_price    volume_1wk
0       US x Iran permanent peace deal by May 31, 2026? 2026-05-31 00:00:00+00:00             0.060  3.941003e+07
1      US x Iran permanent peace deal by June 30, 2026? 2026-05-31 00:00:00+00:00             0.940  1.439399e+07
2       US x Iran permanent peace deal by June 7, 2026? 2026-06-07 00:00:00+00:00             0.001  1.293185e+07
3      US x Iran permanent peace deal by June 15, 2026? 2026-06-15 00:00:00+00:00             0.846  4.781544e+07
4      US x Iran permanent peace deal by July 31, 2026? 2026-07-31 00:00:00+00:00             0.950  2.967144e+06
5    US x Iran permanent peace deal by August 31, 2026? 2026-08-31 00:00:00+00:00             0.959  8.296762e+05
6   US x Iran permanent peace deal by October 31, 2026? 2026-10-31 00:00:00+00:00             0.969  5.548952e+05
7  US x Iran permanent peace deal by December 31, 2026? 2026-12-31 00:00:00+00:00             0.973  4.657350e+06
```

One event, one market per deadline. For ended markets, `last_trade_price` is the last price before the end (note the June 30 market's `end_date`: a catalog error worth a second look).

Backup A5: a first statistic, the prices of live markets

```
prices = ask("""
    SELECT last_trade_price FROM markets
    WHERE end_date > now()
      AND volume_1wk > 10000
      AND last_trade_price IS NOT NULL
""")
prices.last_trade_price.astype(float).plot.hist(bins=40
```

Most live markets sit near 0: events with dozens of outcomes make nearly every outcome a long shot.



Backup A6 and A7: save a table; peek into one hour of order-book data

```
top.to_csv("top_markets.csv", index=False)      # opens in Excel or Google Sheets
```

```
saved top_markets.csv with 10 rows
```

```
book(f"SELECT * FROM '{BASE}/events-2026-09-29T12.parquet' LIMIT 5")
```

	ts_ms	asset_id	market	side	price	size
0	1790683200000	513564763935194504903488 76…	0x9616c1111bbb24c29a0af5…	BUY	0.22	4617.10
1	1790683200000	519897276833433193675188 09…	0x9616c1111bbb24c29a0af5…	SELL	0.78	4617.10
2	1790683200011	513564763935194504903488 76…	0x9616c1111bbb24c29a0af5…	BUY	0.22	4628.10
3	1790683200011	519897276833433193675188 09…	0x9616c1111bbb24c29a0af5…	SELL	0.78	4628.10
4	1790683200021	513564763935194504903488 76…	0x9616c1111bbb24c29a0af5…	SELL	0.76	2070.83

Backup A7: count the level changes per side in one hour

```
book(f"""
    SELECT side, count(*) AS level_changes, round(avg(price), 3) AS average_price
    FROM '{BASE}/events-2026-09-29T12.parquet'
    GROUP BY side
""")
```

```
   side  level_changes  average_price
0  SELL         832599          0.544
1   BUY         832599          0.456
```

- Exactly as many changes on each side, and average prices that sum to one dollar: every change to a Yes book is mirrored in the No book of the same market.
- 1.67 million rows for one hour, counted in about four seconds without downloading the file.

Backup A8: one market's wallets, then one wallet across five days

```
book(f"""
    SELECT wallet, round(sum(dollars)) AS volume, count(*) AS trades, count(DISTINCT outcome) AS sides
    FROM '{BASE}/wallet_trades/trades-2026-10-05.parquet'
    WHERE market = 'atp-djokovi-medvede-2026-10-05'
    GROUP BY wallet ORDER BY volume DESC LIMIT 5
""")
```

wallet	volume	trades	sides
0xd512a29e995addc51005fc9b80e7469ce7e20dce	198,159	8	1
0xf65a95bcd2e016e2a736adb63e8c9746018132bd	175,088	19	1
0xe0446ec594b827d5934e11420318d307e327337f	55,632	86	2
0xb73be06da617714a70c0f40b267db57d64379803	28,452	28	2
0x83353b9519001611c5342e471efc427627896710	27,600	1	1

```
w = top.wallet[0]                                  # the biggest wallet above; same pseudonym in every file
days = [f"'{BASE}/wallet_trades/trades-2026-10-0{d}.parquet'" for d in range(1, 6)]
book(f"SELECT market, min(time) AS first_fill, count(*) AS fills, round(sum(dollars)) AS dollars "
     f"FROM read_parquet([{', '.join(days)}]) WHERE wallet = '{w}' GROUP BY market ORDER BY dollars DESC")
```

market	first_fill	fills	dollars
atp-djokovi-medvede-2026-10-05	2026-10-05 04:47:55-07:00	8	198,159
atp-rai-lomakin-2026-10-03	2026-10-03 13:50:40-07:00	1	30,074
atp-bayldon-ichikaw-2026-10-04	2026-10-04 00:33:28-07:00	1	26,513
atp-kikuchi-ogura-2026-10-04	2026-10-04 00:46:41-07:00	1	25,793
atp-purcell-cuyper-2026-10-04	2026-10-03 22:37:22-07:00	1	9,000
atp-matsud-betov-2026-10-04	2026-10-04 00:44:19-07:00	1	7,652

Backup, demo 1–2: load the wallet table, set the bots aside

```
W = pd.read_csv("data/wallet_features.csv", index_col="wallet")
FEATS = ["position_anomaly_ratio", "market_notional_ratio", "avg_buy_price", "concentration", "wallet_age_days", "unique_markets"]
print(f"{len(W):,} wallets, {int(W.confirmed.sum())} of them publicly confirmed insiders")

W["is_bot"] = (W.unique_markets >= 500) | (W.trade_count >= 2000)
H = W[~W.is_bot].copy()
print(f"bots set aside: {W.is_bot.sum()}   humans to score: {len(H):,}")
```

```
55,952 wallets, 19 of them publicly confirmed insiders
bots set aside: 284   humans to score: 55,668
```

Backup, demo 3–4: fit the forest, read the top of the list

```
from sklearn.ensemble import IsolationForest
X = H[FEATS].to_numpy(float)                          # missing values -> column median (see notebook)
forest = IsolationForest(n_estimators=200, contamination=0.05, random_state=42, n_jobs=-1).fit(X)
H["raw"] = forest.decision_function(X)                # negative = anomalous
H["score"] = 100 / (1 + np.exp(10 * H.raw))           # 0..100
H["rank"] = H.raw.rank(method="first").astype(int)    # 1 = most anomalous
H = H.sort_values("rank"); top1 = int(np.ceil(0.01 * len(H)))
print(f"top 1% = {top1} wallets. Confirmed insiders among them: {int(H.iloc[:top1].confirmed.sum())}")
```

```
top 1% = 557 wallets. Confirmed insiders among them: 1

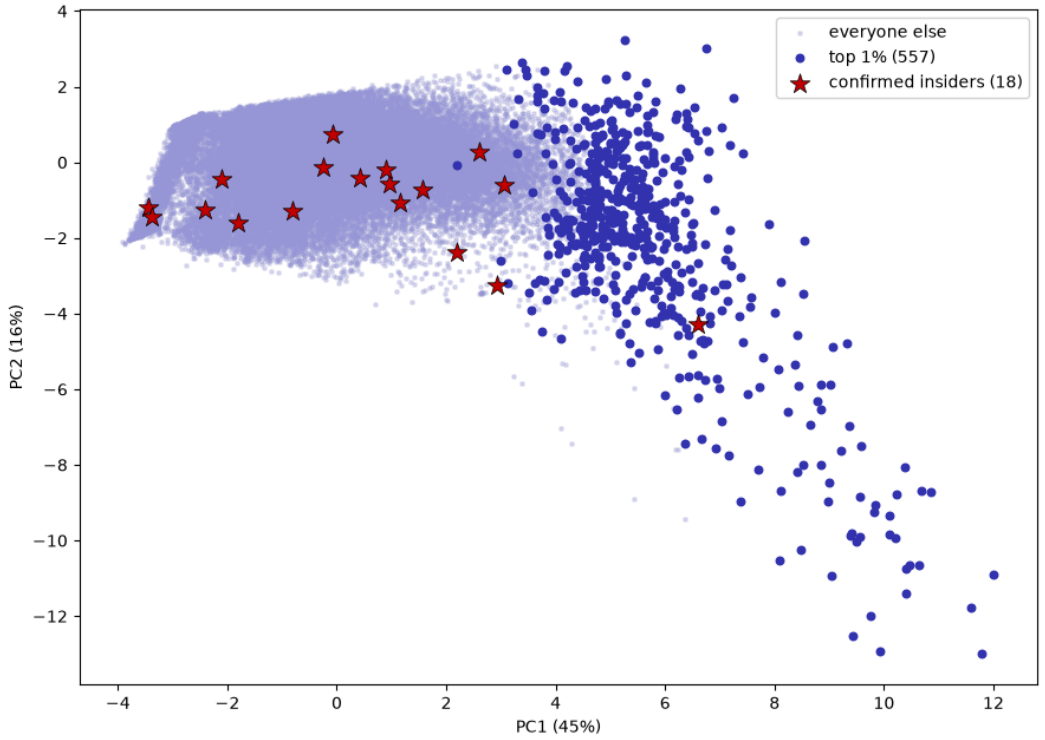
wallet         score  insider  pos_ratio  mkt_share  price  conc   age_d  markets   total_$
w_3c83e74381  91.988           19.211     0.176      0.580  0.094  329.2    224      5,766
w_592a48ad5c  91.827           23.117     0.221      0.655  0.128  330.1    186     28,153
w_4fad99896f  91.826           23.648     0.169      0.913  0.110  346.5    264    247,837
w_a085741c51  91.584           36.276     0.288      0.473  0.157  171.8    233        909
w_8577bb2d2f  91.114           30.830     0.181      0.932  0.318  274.4    135     12,623
w_a8a52f1188  90.983           21.672     0.086      0.490  0.099  279.1    221     50,937
w_cdb76c15a1  90.938           15.124     0.344      0.899  0.179  357.1    137    491,129
w_420bee7db7  90.663            8.652     0.616      0.966  0.168  326.5    139    316,369
w_06c7a3672b  90.592            8.517     0.601      0.980  0.061  264.4    157    329,192
w_cf61d5d09a  90.216           50.003     0.046      0.681  0.238  349.7    218     25,304
```

Backup, demo 5–6: where the confirmed insiders rank; precision at k

wallet	rank	score	pos_ratio	mkt_share	price	conc	age_d	markets	total_$	percentile
w_2a43266bd8	264	81.759	4.063	0.261	0.897	0.147	300.5	55	675,487	0.47
w_bc876cca82	1301	63.735	0.607	0.151	0.788	0.251	41.9	22	359,436	2.34
w_bae5b984cf	2109	54.765	1.575	0.093	0.562	0.291	39.8	13	297,221	3.79
w_b484b941ed	2665	50.727	4.645	0.011	0.661	0.579	245.0	26	489,359	4.79
w_651e7829e2	2741	50.277	0.756	0.034	0.815	0.302	241.9	8	105,116	4.92
w_64ac068f22	3670	45.168	1.056	0.013	0.450	0.412	260.4	10	26,935	6.59
w_d1a074f023	4238	42.609	0.773	0.026	0.727	0.415	208.9	6	65,326	7.61
w_ce629884a4	4400	42.041	2.093	0.006	0.734	0.108	120.2	38	3,812	7.90
w_45933e3f6b	5647	38.174	0.450	0.040	0.533	0.577	9.5	3	42,668	10.14
w_399ef66fe6	7801	33.051	3.089	0.019	0.827	0.803	8.3	4	16,210	14.01
w_b61c7eb379	8480	31.901	0.693	0.004	0.152	0.694	8.7	2	5,234	15.23
w_2023847f0c	9559	30.285	0.333	0.000	0.200	1.000	0.0	1	18,136	17.17
w_2c4d5f68c4	11539	28.135	0.827	0.003	0.234	0.747	0.0	2	14,392	20.73
w_efad88a8e8	13682	26.251	0.600	0.000	0.100	1.000	0.0	1	1,865	24.58
w_cd38860893	16055	24.497	0.210	0.004	0.585	0.706	2.3	2	75,327	28.84
w_ea3af901bf	18532	22.905	2.758	0.000	0.416	0.813	11.5	7	10,787	33.29
w_bf68f8e7eb	23399	20.393	0.721	0.002	0.822	0.198	5.7	9	46,029	42.03
w_89b41a54ca	33136	17.112	1.539	0.001	0.793	0.212	7.8	10	38,445	59.52

top_percent	wallets_flagged	of	forest_catches	biggest_spenders_catch	chance
0.5	279	18	1	4	0.09
1.0	557	18	1	5	0.18
2.5	1392	18	2	9	0.45
5.0	2784	18	5	13	0.90
10.0	5567	18	8	16	1.80

Backup, demo 8–9: the population in 2-D; what the misses share




```
missed = lab[lab["rank"] > int(np.ceil(0.05 * n))]
missed[["rank", "unique_markets", "wallet_age_days",
        "trade_count", "concentration"]]
```

wallet	rank	markets	age_days	trades	concentration
w_64ac068f22	3670	10	260.36	96	0.41
w_d1a074f023	4238	6	208.95	199	0.42
w_ce629884a4	4400	38	120.23	310	0.11
w_45933e3f6b	5647	3	9.53	136	0.58
w_399ef66fe6	7801	4	8.29	31	0.80
w_b61c7eb379	8480	2	8.68	81	0.69
w_2023847f0c	9559	1	0.02	122	1.00
w_2c4d5f68c4	11539	2	0.05	75	0.75
w_efad88a8e8	13682	1	0.00	7	1.00
w_cd38860893	16055	2	2.25	155	0.71
w_ea3af901bf	18532	7	11.46	193	0.81
w_bf68f8e7eb	23399	9	5.69	134	0.20
w_89b41a54ca	33136	10	7.80	139	0.21

Backup: what one trade looks like on the chain

Transaction 0xd209e52b…21ce2728 · Polygon block 94,682,902 · 2026-09-30 00:51:33 UTC

sent by	0xfee9c464…bf7ec9 *a Polymarket relayer: it pays the gas, not the trader*
to	0xe1111800…7b996b *the exchange contract that matches orders*
event 1	0x1e31c6b0… → exchange 5.192990 pUSD *taker pays*
event 2	0xd1efe8a5… → exchange 4.615385 pUSD *maker pays*
event 10	PositionSplit on 0xacec7e5b… 9.615385 *$9.62 becomes 9.62 Yes + 9.62 No*
event 12	Yes token → 0xd1efe8a5… 9.615385
event 16	No token → 0x1e31c6b0… 9.615385
event 14	0.192990 pUSD → 0x115f48dc… *the fee*
gas used	587,401 · 20 events in total

- A **wallet** is an address. Polymarket gives each user a *proxy* wallet; the trade API reports that one.
- A **token** is a share; its id encodes the market and the outcome. Shares are **minted in pairs**: $1 in, one Yes and one No out. Slide 6 is a smart contract.
- Public, forever. The chain is the ground truth the APIs summarise.

Reading the chain yourself

Any public Polygon node: `eth_getTransactionReceipt(hash)`. Or paste the hash into polygonscan.com.

Decoded from the transaction receipt of the trade on slide 22. pUSD is the dollar token Polymarket settles in; 6 decimals. Addresses shortened. **[Polygon public RPC, Sep 30 2026]**